```
#!/usr/bin/env bash
# GS Linux — assistente para um laboratório Debian/Rocky Linux.
# Uso: bash gs-linux.sh --demo | sudo bash gs-linux.sh
# Cada função representa uma etapa; as tarefas extras ficam no final do arquivo.

set -Eeuo pipefail
umask 022

VERSION=1.2.5-GS
SCRIPT_DIR=$(cd -- "$(dirname -- "${BASH_SOURCE[0]:-$0}")" 2>/dev/null && pwd) || SCRIPT_DIR=.
# Cópia local do template (pasta ao lado do script) e hashes conhecidos dos arquivos.
# Os hashes ficam AQUI, dentro do .sh: valem tanto para o download quanto para a cópia local.
TEMPLATE_DIR=${GS_TEMPLATE_DIR:-$SCRIPT_DIR/template} TEMPLATE_SOURCE=''
TEMPLATE_SHA256=(
    '0417dd0f16e8db738a8dd20ce57ad7a6867ae163971684ee446c846c075607be  index.html'
    '058eafd647eb2e24597d598659f4a963d928d92bfd33da5ce730c39b80f945ff  css/styles.css'
    '8971cc202dc06b18e7bde4174ecdcbde1454a316fb018af253c8f5bc05059d00  js/scripts.js'
    '8b1060f193768c55bc48dede8ba8da9be6c93f8e6dcae946e36f1a8956b7269b  assets/favicon.ico'
    'e8b857747301fcd4c885b3216544b18bbf60328aec2c34da614787346cf40282  LICENSE'
    '9520018fa5d81f4e4dc9d06afb576f90cbbaba209cfcc6cb60e1464647f7890b  js/bootstrap.bundle.min.js'
)
MODE=menu
DOMAIN='' SERVER_IP='' PLATFORM='' DNS_SERVICE='' WEB_SERVICE=''
DNS_CONF='' ZONE_FILE='' DNS_GROUP='' WEB_CONF='' WEB_LINK=''
DNS_ROLE=primary PRIMARY_IP='' SECONDARY_IP='' WEB_IP='' SERVER_IFACE=''
REVERSE_NETWORK='' REVERSE_ZONE='' REVERSE_PREFIX='' REVERSE_FILE='' ZONE_DIR=''
LEGACY_STATE=0
declare -a LOCAL_IPS=() LOCAL_IFACES=() PTR_IP=() PTR_NAME=()
WEB_ROOT=/var/www/gs-linux
STATE_DIR=/var/lib/gs-linux
BACKUP='' WORK='' LOG='' TRANSACTION=0
DEBIAN_CODENAME='' APT_BACKUP='' APT_REPAIR_PENDING=0
declare -a APT_PATHS=() APT_EXISTS=()
DNS_WAS_ACTIVE=0 WEB_WAS_ACTIVE=0 DNS_WAS_ENABLED=0 WEB_WAS_ENABLED=0
declare -a REC_NAME=() REC_TYPE=() REC_VALUE=() REC_PRIORITY=()
declare -a SAVED_PATHS=() SAVED_EXISTS=() FW_ADDED=()
RED='' GREEN='' YELLOW='' CYAN='' BLUE='' MAGENTA='' BOLD='' RESET='' MUTED=''
UI_WIDTH=76

colors() {
    local columns=${COLUMNS:-80}
    if [[ -t 1 ]] && command -v tput >/dev/null; then columns=$(tput cols 2>/dev/null) || columns=80; fi
    [[ $columns =~ ^[0-9]{1,4}$ ]] || columns=80
    UI_WIDTH=$((10#$columns-4))
    (( UI_WIDTH <= 76 )) || UI_WIDTH=76
    (( UI_WIDTH >= 20 )) || UI_WIDTH=20
    if [[ -t 1 && ${TERM:-dumb} != dumb && -z ${NO_COLOR+x} ]]; then
        RED=$'\e[91m'; GREEN=$'\e[92m'; YELLOW=$'\e[93m'
        CYAN=$'\e[36m'; BLUE=$'\e[94m'; MAGENTA=$'\e[95m'; BOLD=$'\e[1m'; RESET=$'\e[0m'
        MUTED=$'\e[90m'
    fi
}
ui_rule() {
    local line
    printf -v line '%*s' "$UI_WIDTH" ''
    printf '  %s%s%s\n' "$MUTED" "${line// /-}" "$RESET"
}
# Quebra apenas os textos da interface por palavras; não altera respostas ou arquivos.
ui_wrap() {
    local offset=$1 text=$2 width line word paragraph padding first=1
    local -a words=()
    width=$((UI_WIDTH-offset)); (( width >= 16 )) || width=16
    printf -v padding '%*s' "$((offset+2))" ''
    while IFS= read -r paragraph; do
        if (( ! first )); then printf '%s' "$padding"; fi
        first=0; line=''
        read -r -a words <<<"$paragraph"
        for word in "${words[@]}"; do
            if [[ -n $line ]] && (( ${#line}+${#word}+1 > width )); then
                printf '%s\n%s' "$line" "$padding"; line=$word
            else line+=${line:+ }$word; fi
        done
        printf '%s\n' "$line"
    done <<<"$text"
}
ui_message() {
    printf '  %s%s%s ' "$1" "$2" "$RESET"
    ui_wrap "$((${#2}+1))" "$3"
}
ui_group() { printf '\n  %s%s%s\n' "$BLUE$BOLD" "$*" "$RESET"; }
ui_detail() { printf '  %s%-14s%s %s%s%s\n' "$MUTED" "$1" "$RESET" "$BOLD" "$2" "$RESET"; }
ui_step() { heading "PASSO $1 DE 5  /  $2"; }
info() { ui_message "$CYAN" '[INFO]' "$*"; }
ok() { ui_message "$GREEN$BOLD" '[ OK ]' "$*"; }
warn() { ui_message "$YELLOW$BOLD" '[AVISO]' "$*"; }
die() { { printf '\n'; ui_message "$RED$BOLD" '[ERRO]' "$*"; } >&2; exit 1; }
```

```
heading() { printf '\n'; ui_rule; printf '  %s%s%s\n' "$CYAN$BOLD" "$*" "$RESET"; ui_rule; }
menu_item() { printf '  %s[ %s ]%s  ' "$CYAN$BOLD" "$1" "$RESET"; ui_wrap "$((${#1}+6))" "$2"; }
explain() { ui_message "$BLUE" '[ENTENDA]' "$*"; }
pause_reading() { ask 'Enter para voltar ao menu'; }
banner() {
    printf '\n%s' "$CYAN$BOLD"
    if (( UI_WIDTH >= 54 )); then
        cat <<'ART'
    ____ ____       _     ___ _   _ _   ___  __
   / ___/ ___|     | |   |_ _| \ | | | | \ \/ /
  | |  _\___ \     | |    | ||  \| | | | |\  /
  | |_| |___) |    | |___ | || |\  | |_| |/  \
   \____|____/     |_____|___|_| \_|\___//_/\_\
ART
    else printf '  GS / LINUX\n'; fi
    printf '%s\n' "$RESET"
    printf '  %sASSISTENTE DA GS%s  %sv%s%s\n' "$BOLD" "$RESET" "$MUTED" "$VERSION" "$RESET"
    printf '  %sDebian + Rocky Linux  /  DNS + Web%s\n' "$BLUE" "$RESET"
    printf '  %s' "$MUTED"; ui_wrap 0 'Aprenda. Configure. Comprove. Use em uma VM de laboratório.'; printf '%s'
"$RESET"
}
help_text() {
    cat <<'HELP'
Uso:
  bash gs-linux.sh               Abre o menu: aprender, treinar, instalar, comprovar.
  bash gs-linux.sh --learn       Glossário e exemplos, sem Internet ou root.
  bash gs-linux.sh --tutor       Orientador por assunto para os pedidos da prova.
  bash gs-linux.sh --demo        Treina os menus sem alterar a máquina.
  sudo bash gs-linux.sh --install Instala e configura DNS + Apache.
  sudo bash gs-linux.sh --status Mostra evidências reais de DNS, Web e sistema.
  sudo bash gs-linux.sh --report Salva essas evidências em um arquivo de texto.
  sudo bash gs-linux.sh --check  Testa a instalação já realizada.
  bash gs-linux.sh --help        Mostra esta ajuda.

Compatibilidade prevista: Debian 11/12/13 e Rocky Linux 8/9/10, com systemd.
Defina NO_COLOR=1 para desativar cores. Não precisa instalar nada para a demo.
A instalação requer Internet, privilégios de root e uma VM com IPv4 estável.
HELP
}

# O texto digitado nunca é executado como comando (não usamos eval).
ask() {
    local prompt=$1 default=${2:-} answer
    printf '\n  %s' "$BOLD"; ui_wrap 0 "$prompt"; printf '%s' "$RESET"
    if [[ -n $default ]]; then
        printf '  %sEnter para usar:%s %s%s%s\n' "$MUTED" "$RESET" "$CYAN$BOLD" "$default" "$RESET"
    fi
    printf '  %s> %s' "$CYAN$BOLD" "$RESET"
    if ! IFS= read -r answer; then die 'Entrada encerrada. Execute em um terminal interativo.'; fi
    answer="${answer#"${answer%%[![:space:]]*}"}"
    answer="${answer%"${answer##*[![:space:]]}"}"
    REPLY=${answer:-$default}
}
# O guia faz parte deste arquivo: continua disponível copiando apenas o .sh.
lesson() {
    local topic=$1 domain=${DOMAIN:-aluno.fiap} server=${SERVER_IP:-192.168.56.10}
    case $topic in
        dns)
            heading 'DNS e BIND — encontrar uma máquina pelo nome'
            explain 'IP é um endereço de rede. DNS permite consultar um nome e obter informações, como seu IP.'
            cat <<EOF
Exemplo: você consulta $domain e o DNS responde $server.
BIND é o programa que responde às consultas DNS. Seu serviço se chama named.
Uma ZONA é o conjunto de registros de um domínio. DOMÍNIO é o nome, como aluno.fiap.
Este script cria um DNS AUTORITATIVO: ele conhece a zona que você cadastrou.
Não é um resolvedor geral da Internet; a recursão está desligada.

PARA MOSTRAR AO PROFESSOR:
  dig @$server $domain A
O @ indica qual DNS consultar. Procure status: NOERROR e a seção ANSWER.
O sinalizador aa significa resposta autoritativa. NXDOMAIN significa nome inexistente.
Timeout significa que a resposta não chegou; pode ser serviço, IP, rede ou firewall.

NA ATIVIDADE DO PROFESSOR:
  Primário (master): cria os registros e envia as zonas ao secundário autorizado.
  Secundário (slave): recebe cópias; não edite os arquivos recebidos nessa VM.
  Primeiro instale o primário e informe o IP reservado para a segunda VM.
  Depois, na segunda VM, escolha Secundário e informe o IP do primário.
  Use o MESMO domínio e a MESMA rede /24 nas duas instalações.
  No perfil de duas VMs, www aponta para a segunda VM; intranet é apelido de www.

Zona direta: nome → IP. Zona reversa: IP → nome, usando registros PTR.
Exemplo: 192.168.56.0/24 → 56.168.192.in-addr.arpa.
O script gera PTRs dos registros A que pertencem à rede /24 escolhida.
Um A externo, como webmail → 10.200.200.200, fica fora dessa zona reversa.
```

```
PARA PROVAR A REPLICAÇÃO:
  Consulte o SOA nos dois IPs: os seriais devem coincidir.
  Acrescente webmail A 10.200.200.200 no primário, mantendo os outros registros.
  Reaplicar aumenta o serial automaticamente; consulte webmail no secundário.
  O painel 5 mostra SOAs, arquivos recebidos, nslookup e logs do BIND.

EXPLICAÇÃO EM UMA FRASE:
  "Configurei o BIND para responder pela zona $domain e consultei esse DNS diretamente."
EOF
            ;;
        records)
            heading 'Registros DNS — qual deles usar?'
            cat <<EOF
A     Nome → IPv4. Ex.: loja.$domain → $server.
      Use quando o professor pedir: "aponte este nome para este IP".
CNAME Apelido → outro nome. Ex.: intranet.$domain → www.$domain.
      Não recebe IP. Neste script, o destino precisa ter um A na mesma zona.
MX    Domínio → nome do servidor de e-mail. Ex.: $domain → mail.$domain.
      Crie antes o A de mail. Prioridade 10 é preferida a 20. MX não instala e-mail.
TXT   Texto associado a um nome. Ex.: "Trabalho da GS". Não aponta para uma máquina.
NS    Nome do servidor responsável pela zona. Aqui: ns1.$domain.
SOA   Informações de controle da zona. Inclui o SERIAL, que identifica sua versão.
PTR   IPv4 → nome. É gerado automaticamente para cada A dentro da rede /24 escolhida.
TTL   Tempo em segundos pelo qual uma resposta pode ficar em cache.
@     Dentro do arquivo da zona: o próprio domínio. No dig: o servidor consultado.

No menu, digite loja: o script completa .$domain para você.
Um nome com CNAME não pode ter também A, MX ou TXT. O script impede esse conflito.
O ponto final em mail.$domain. indica um nome completo no arquivo da zona.
Sem ele, o BIND pode acrescentar o domínio de novo. O script coloca o ponto.
www já é A. Use a opção 7 para mudar seu IP; intranet continuará apontando para www.
Você pode remover os registros da base; ns1 é protegido porque sustenta SOA e NS.
Para mudar o tipo, remova o registro e cadastre outro; remova antes seus dependentes.

SOA DA ATIVIDADE (valores em segundos):
  SERIAL: AAAAMMDDnn, por exemplo 2026110400; cada reaplicação aumenta o contador.
  REFRESH 28800: intervalo normal para o secundário procurar atualizações (8 horas).
  RETRY 14400: espera para tentar novamente se a consulta falhar (4 horas).
  EXPIRE 86400: limite sem atualizar antes de deixar de servir a cópia (1 dia).
  MINIMUM 600: tempo de cache negativo, como resposta de nome inexistente (10 minutos).
  NOTIFY avisa o secundário de alterações; ele não precisa esperar o REFRESH.

PARA MOSTRAR:
  dig @$server loja.$domain A
  dig @$server www.$domain A
  dig @$server intranet.$domain CNAME
  dig @$server -x $server
  dig @$server $domain MX
  dig @$server $domain TXT

AAAA é para IPv6 e SRV para descoberta de serviços. Esses dois não estão implementados.
Zona reversa neste perfil exige /24. Não atende delegação reversa de outros prefixos.
EOF
            ;;
        web)
            heading 'Apache, HTTP e template — entregar a página'
            explain 'DNS encontra o endereço. Apache recebe a requisição HTTP e entrega o HTML.'
            cat <<EOF
Apache é o servidor Web: serviço apache2 no Debian e httpd no Rocky.
HTML é o conteúdo; CSS controla a aparência; JavaScript fornece interações.
Template é um modelo pronto desses arquivos. Aqui usamos Bare / Start Bootstrap.
DocumentRoot é a pasta publicada: $WEB_ROOT.
VirtualHost associa nomes como $domain a um site do Apache.
HTTP usa TCP/80. HTTPS usa TCP/443 e certificado; HTTPS não está configurado aqui.

PARA MOSTRAR:
  curl --noproxy '*' -I http://$server/
  curl --noproxy '*' --resolve $domain:80:$server http://$domain/
O primeiro mostra os cabeçalhos: espere HTTP 200. O segundo pede o HTML pelo nome.
--resolve configura o endereço só para esse curl: ele NÃO comprova o DNS do cliente.
Mostre também a consulta dig e abra o site no navegador de outro computador.

SE O PROFESSOR PEDIR "OUTRO SITE":
  Separe outra pasta e outro VirtualHost; separe também o registro DNS do novo nome.
  O assistente atual gerencia um único site. Outra execução substitui esse site.
EOF
            ;;
        network)
            heading 'IP, máscara, gateway e portas'
            cat <<'EOF'
IPv4: endereço como 192.168.56.10. Use o da VM, não o do seu computador hospedeiro.
Máscara/CIDR: define a rede. /24 corresponde a 255.255.255.0.
Exemplo: 192.168.56.0/24 tem, normalmente, hosts .1 até .254; .255 é broadcast.
Gateway: roteador usado para alcançar outras redes. Não é necessariamente o DNS.
Interface: conexão da VM, com nome como enp0s3, ens33 ou eth0.
Porta: identifica um serviço dentro de um IP. O mesmo número pode existir em TCP e UDP.
```

```
DNS: UDP/53 e TCP/53. Web deste projeto: TCP/80. SSH: TCP/22.
DHCPv4: servidor UDP/67, cliente UDP/68.

CONSULTAS QUE NÃO ALTERAM A REDE:
  ip -4 addr       # endereços e interfaces
  ip route         # rotas e gateway padrão
  ss -lntup        # portas em escuta; sudo mostra mais detalhes dos processos

ping testa resposta ICMP. Um ping funcionando NÃO prova que DNS ou HTTP funcionam.
Uma porta em escuta NÃO prova que outro computador consegue chegar até ela.
Se o IP da VM mudar por DHCP, a configuração DNS salva pode ficar desatualizada.

VIRTUALBOX COM DUAS PLACAS:
  Host-Only: rede compartilhada pelas VMs do laboratório, normalmente 192.168.56.x.
  NAT: saída para Internet, frequentemente 10.0.2.15. Duas VMs NAT podem exibir esse
  mesmo IP sem conseguirem conversar por ele. Escolha o IP Host-Only no assistente.
  A lista prioriza 192.168.56.x; confira os endereços do enunciado antes de aceitar.
  Configure antes o IP fixo na interface Host-Only. O script não altera a rede da VM.
EOF
            ;;
        dhcp)
            heading 'DHCP — distribuir a configuração de rede'
            explain 'DHCP entrega IP, máscara, gateway e DNS automaticamente aos clientes.'
            cat <<'EOF'
DNS responde a perguntas sobre nomes; DHCP fornece configuração de rede.
LEASE é a concessão de um endereço por um período. RESERVA associa um IP a um cliente,
geralmente identificado pelo MAC, que é o identificador da interface de rede.

EXEMPLO DE ESTUDO (não copie sem conferir a rede do enunciado):
  Rede: 192.168.56.0/24       Servidor DNS: 192.168.56.10
  Faixa dinâmica: .100 a .150  Gateway: deve ser o roteador REAL dessa rede
  O IP estável do servidor fica fora da faixa dinâmica.

SE PEDIREM DHCP, LEVANTE:
  1. Interface e rede de laboratório; máscara; IP do servidor.
  2. Faixa de endereços, reservas e tempo de concessão.
  3. Gateway real (se existir) e IP do DNS a fornecer aos clientes.
  4. Implementação pedida em aula: por exemplo, Kea ou ISC DHCP.
  5. Arquivo, sintaxe, serviço e permissões da implementação/distribuição escolhida.

COMO COMPROVAR:
  Em uma VM cliente de teste, obtenha uma concessão; mostre seu IP, rota e DNS.
  No servidor, mostre a concessão e o log do serviço, além da porta UDP/67.
  Ver apenas um processo ativo não prova que o cliente recebeu uma concessão.

ESTADO DESTE PROJETO: DHCP ainda não está implementado. Este guia não o instala.
Ative um DHCP apenas na rede isolada autorizada pelo professor: um segundo servidor
na rede errada pode entregar endereços incorretos a outros computadores.
EOF
            ;;
        firewall)
            heading 'Firewall e SELinux — duas verificações diferentes'
            cat <<'EOF'
Firewall controla tráfego de rede. Neste projeto precisamos de DNS TCP/UDP 53 e HTTP TCP 80.
firewalld organiza permissões por ZONA; confira a zona da interface usada pela VM.
Regras runtime valem agora; regras permanent ficam salvas para a próxima inicialização.
UFW é outro gerenciador de firewall, frequente no Debian.

CONSULTAS:
  sudo firewall-cmd --get-active-zones
  sudo firewall-cmd --zone=public --list-all
  sudo firewall-cmd --permanent --zone=public --list-all
Troque public pela zona real da sua interface. No UFW: sudo ufw status verbose.

SELinux controla o acesso de processos a recursos, como arquivos. No Rocky, pode
impedir Apache de ler uma pasta com contexto incorreto, mesmo com a porta liberada.
  getenforce                    # Enforcing aplica as regras; Permissive só registra
  ls -Zd /var/www/gs-linux      # contexto da pasta do site
  sudo ausearch -m AVC -ts recent  # negativas recentes, se audit estiver instalado

Não desligue o firewall ou SELinux para "provar" que a configuração funciona.
O painel consulta o estado real. Regras locais não comprovam passagem pelo roteador
ou firewall do provedor: faça também uma requisição a partir de outra máquina.
EOF
            ;;
        services)
            heading 'Serviços, inicialização e logs'
            cat <<EOF
Serviço é um programa administrado pelo sistema. systemd é o gerenciador usado aqui.
ACTIVE significa que está rodando agora. ENABLED significa configurado para o boot.
Um serviço pode estar ativo e desabilitado no boot, ou habilitado e parado.

CONSULTAS:
  systemctl is-active named
  systemctl is-enabled named
```

```
  systemctl status named --no-pager
  journalctl -u named -n 30 --no-pager
No Apache use apache2 (Debian) ou httpd (Rocky) no lugar de named.

Antes de reiniciar, valide a configuração:
  sudo named-checkconf -z
  sudo apache2ctl configtest    # Debian
  sudo httpd -t                 # Rocky

restart reinicia; reload recarrega quando o serviço suporta; enable configura o boot.
Esses comandos alteram o sistema. status, is-active e journalctl só consultam.
Logs são o histórico: procure a mensagem e o horário da falha, não só a palavra ERROR.
O instalador guarda seus logs em $STATE_DIR e backups em /var/backups/gs-linux.
EOF
            ;;
        exam)
            heading 'Possíveis exercícios — roteiro de estudo, não previsão'
            cat <<'EOF'
PEDIDO                          ONDE COMEÇAR                          COMO COMPROVAR
Novo nome/IP                    Menu de registros: tipo A             dig @IP nome A
Apelido de um site               A do destino e depois CNAME           dig @IP apelido CNAME
Servidor de e-mail no DNS        A do mail + MX com prioridade         dig @IP dominio MX
Texto no domínio                TXT                                   dig @IP dominio TXT
Publicar outro conteúdo         Template / DocumentRoot               curl + navegador
Liberar acesso ao site          Firewall na interface correta         Regra + teste de outro PC
Iniciar após reiniciar          systemctl enable do serviço           is-enabled + teste após boot
IP automático para clientes     Planejar DHCP em rede isolada         Concessão real no cliente
Outro domínio/site              Nova zona + novo VirtualHost          dig e curl dos dois sites
Resolução reversa /24           Automática para os A locais           dig -x IP @servidor
DNS secundário                 Papel 2 na segunda VM                 SOAs iguais + consulta nova
Backup agendado                  Script de backup + cron/timer         Arquivo gerado + restauração
Usuários/permissões              id, grupos, ls -l; menor privilégio   Teste como aquele usuário

DHCP, outro site, backup agendado e gestão de usuários exigem implementação adicional.
Para qualquer tarefa: anote o pedido → planeje → faça backup → altere → valide → teste.
Leia extra_tasks no código: é o ponto reservado às chamadas das funções da prova.
Não use chmod 777 como solução geral e não teste restauração por cima da única cópia.
EOF
            ;;
    esac
}
learning_menu() {
    while :; do
        heading 'APRENDER — escolha o que ainda não entendeu'
        ui_group 'FUNDAMENTOS DO LABORATÓRIO'
        menu_item 1 'DNS, BIND, zona reversa e primário/secundário'
        menu_item 2 'A, CNAME, MX, TXT, PTR, NS, SOA e TTL'
        menu_item 3 'Apache, HTTP, template e VirtualHost'
        menu_item 4 'IP, máscara, gateway e portas'
        ui_group 'SERVIÇOS, DIAGNÓSTICO E PROVA'
        menu_item 5 'DHCP: o que é e o que o professor pode pedir'
        menu_item 6 'Firewall e SELinux'
        menu_item 7 'Serviços, inicialização e logs'
        menu_item 8 'Possíveis tarefas e como demonstrar cada uma'
        printf '\n'
        menu_item 0 'Voltar / sair do guia'
        ask 'Assunto' 0
        case $REPLY in
            1) lesson dns ;; 2) lesson records ;; 3) lesson web ;; 4) lesson network ;;
            5) lesson dhcp ;; 6) lesson firewall ;; 7) lesson services ;; 8) lesson exam ;;
            0) return ;; *) warn 'Escolha uma opção de 0 a 8.'; continue ;;
        esac
        pause_reading
    done
}
tutor_menu() {
    local question topic topic_title action
    heading 'ORIENTADOR DA PROVA'
    explain 'Roteiro local por palavras-chave: funciona sem Internet e não é uma IA conectada.'
    explain 'Você confirma o assunto. O orientador explica os passos; não executa comandos por você.'
    while :; do
        ask 'O que o professor pediu? Escreva uma frase, ou 0 para sair' 0
        question=${REPLY,,}; [[ $question == 0 ]] && return
        case $question in
            *dhcp*|*concess*|*ip*autom*tic*|*endere*autom*tic*) topic=dhcp ;;
            *revers*|*secund*|*slave*|*prim*rio*|*master*|*transfer*) topic=dns ;;
            *cname*|*registro*|*apelido*|*mx*|*txt*|*soa*|*serial*|*ptr*) topic=records ;;
            *firewall*|*porta*|*selinux*|*ufw*) topic=firewall ;;
            *apache*|*http*|*site*|*template*|*web*) topic=web ;;
            *dns*|*bind*|*dom*nio*|*zona*) topic=dns ;;
            *servi*|*boot*|*reinici*|*log*) topic=services ;;
            *rede*|*gateway*|*m*scara*|*ip*) topic=network ;;
            *) topic=exam ;;
        esac
        case $topic in
            dhcp) topic_title=DHCP ;; dns) topic_title=DNS ;; records) topic_title='Registros DNS' ;;
```

```
            firewall) topic_title='Firewall/SELinux' ;; web) topic_title='Servidor Web' ;;
            services) topic_title='Serviços e logs' ;; network) topic_title='Rede e endereços IP' ;;
            exam) topic_title='Roteiro geral de estudo' ;;
        esac
        info "Assunto sugerido: $topic_title. Uma frase pode envolver mais de um assunto."
        ask 'É esse o assunto? S = sim; N = abrir o guia e escolher' S
        if [[ ${REPLY,,} != s && ${REPLY,,} != sim ]]; then learning_menu; continue; fi
        lesson "$topic"
        menu_item 1 'Como organizar a alteração no script?'
        menu_item 2 'Como provar que funcionou?'
        menu_item 3 'Deu erro: por onde começar?'
        menu_item 0 'Fazer outra pergunta'
        ask 'Próximo passo' 0; action=$REPLY
        case $action in
            1)
                if [[ $topic == dhcp ]]; then
                    ask 'O pedido exige distribuir IPs automaticamente aos clientes? (S/N)' S
                    if [[ ${REPLY,,} == s || ${REPLY,,} == sim ]]; then
                        info 'Anote: implementação exigida, interface isolada, rede/máscara, faixa, gateway, DNS e
tempo de concessão.'
                        info 'Crie uma função específica em extra_tasks; valide a configuração antes de iniciar o
serviço.'
                        warn 'DHCP ainda exige implementação. Este orientador não cria um servidor DHCP.'
                    else explain 'Se for só escolher um IP fixo ou resolver nomes, consulte IP/rede ou DNS no
guia.'; fi
                elif [[ $topic == records || $topic == dns ]]; then
                    info 'Use a instalação para definir a lista completa dos registros. Reaplicar substitui a lista
 anterior.'
                    info 'No código: prompt_record coleta os dados; render_zone gera a zona; verify_services a
testa.'
                else
                    info 'Separe o pedido em uma função; faça backup_path antes de alterar novos arquivos.'
                    info 'Use run para comandos com log, chame a função em extra_tasks e adicione uma verificação
do resultado.'
                    info 'Para Web, leia render_web; para firewall, configure_firewall; para validações,
verify_services.'
                fi ;;
            2)
                info 'No menu principal escolha 5, ou execute: sudo bash gs-linux.sh --status'
                info 'Mostre a evidência específica do assunto acima; uma linha verde sozinha não explica o
resultado.'
                [[ $topic != dhcp ]] || warn 'O painel não testa DHCP: mostre também uma concessão real no cliente
e no servidor.'
                info 'Para guardar a demonstração: sudo bash gs-linux.sh --report' ;;
            3)
                info 'Anote o comando, a mensagem completa e o horário. Verifique primeiro a sintaxe e depois o log
 do serviço.'
                info 'Confira IP/interface, nome consultado, porta, firewall e permissões, nessa ordem.'
                info 'O painel --status continua após falhas e ajuda a localizar a etapa com problema.' ;;
            0) continue ;; *) warn 'Opção desconhecida. Retornando às perguntas.' ;;
        esac
        pause_reading
    done
}
home_menu() {
    while :; do
        heading 'O QUE VOCÊ QUER FAZER?'
        ui_group 'COMECE POR AQUI'
        menu_item 1 'Aprender: glossário e exemplos'
        menu_item 2 'Orientador: o professor pediu...'
        menu_item 3 'Treinar a instalação (demonstração, sem alterações)'
        ui_group 'CONFIGURE SUA VM'
        menu_item 4 'Instalar/configurar DNS + Web nesta VM'
        ui_group 'MOSTRE O RESULTADO'
        menu_item 5 'Comprovar: serviços, portas, DNS, Web e firewall'
        menu_item 6 'Salvar relatório de comprovação em texto'
        printf '\n'
        menu_item 0 'Sair'
        ask 'Escolha uma opção' 0
        case $REPLY in
            1) learning_menu ;; 2) tutor_menu ;; 3) MODE=demo; return ;; 4) MODE=install; return ;;
            5) MODE=status; return ;; 6) MODE=report; return ;; 0) MODE='exit'; return ;;
            *) warn 'Escolha uma opção de 0 a 6.' ;;
        esac
    done
}
valid_ipv4() {
    local value=$1 part
    local -a parts
    [[ $value =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]] || return 1
    IFS=. read -r -a parts <<< "$value"
    for part in "${parts[@]}"; do
        [[ ${#part} -le 3 && ( $part == 0 || $part != 0* ) ]] || return 1
        (( 10#$part <= 255 )) || return 1
    done
}
```

```
valid_hostname() {
    local value=$1 label
    local -a labels
    [[ ${#value} -le 253 && $value =~ ^[a-z0-9.-]+$ && $value != .* && $value != *. && $value != *..* ]] || return
1
    IFS=. read -r -a labels <<< "$value"
    for label in "${labels[@]}"; do
        [[ ${#label} -le 63 && $label != -* && $label != *- ]] || return 1
    done
}
valid_domain() { [[ $1 == *.* && ! $1 =~ ^[0-9.]+$ ]] && valid_hostname "$1"; }
valid_owner() {
    local value=$1 type=$2
    [[ $value == @ ]] && return 0
    # TXT pode usar nomes como _dmarc. Os demais seguem nomes de host.
    if [[ $type == TXT ]]; then value=${value//_/a}; fi
    valid_hostname "$value" && (( ${#value} + ${#DOMAIN} + 1 <= 253 ))
}
valid_priority() { [[ $1 =~ ^[0-9]{1,5}$ ]] && (( 10#$1 <= 65535 )); }
valid_txt() { [[ -n $1 && ${#1} -le 200 && ! $1 =~ [^[:print:]] ]]; }

select_platform() {
    local id=$1 version=${2%%.*}
    case "$id:$version" in
        debian:11|debian:12|debian:13)
            case $version in 11) DEBIAN_CODENAME=bullseye ;; 12) DEBIAN_CODENAME=bookworm ;; 13)
DEBIAN_CODENAME=trixie ;; esac
            PLATFORM=debian; DNS_SERVICE=named; WEB_SERVICE=apache2; DNS_GROUP=bind
            DNS_CONF=/etc/bind/named.conf; ZONE_FILE=/var/lib/bind/gs-linux/db.zone
            WEB_CONF=/etc/apache2/sites-available/000-000-gs-linux.conf
            WEB_LINK=/etc/apache2/sites-enabled/000-000-gs-linux.conf ;;
        rocky:8|rocky:9|rocky:10)
            PLATFORM=rocky; DNS_SERVICE=named; WEB_SERVICE=httpd; DNS_GROUP=named
            DNS_CONF=/etc/named.conf; ZONE_FILE=/var/named/gs-linux/db.zone
            WEB_CONF=/etc/httpd/conf.d/00-gs-linux.conf; WEB_LINK='' ;;
        *) return 1 ;;
    esac
}
detect_platform() {
    [[ -r /etc/os-release ]] || die 'Não encontrei /etc/os-release.'
    # Arquivo fornecido pelo sistema operacional, nunca dados digitados pelo usuário.
    # shellcheck source=/dev/null
    . /etc/os-release
    if ! select_platform "$ID" "$VERSION_ID"; then
        if [[ $MODE == demo ]]; then
            warn 'Sistema fora da lista; a demonstração usará os caminhos do Debian.'
            select_platform debian 12
        else die "Sistema não previsto: $PRETTY_NAME. Use Debian 11–13 ou Rocky 8–10."; fi
    fi
    info "Sistema: $PRETTY_NAME | DNS: BIND | Web: Apache ($WEB_SERVICE)."
}
detect_ip() {
    local candidate=''
    if command -v ip >/dev/null; then
        candidate=$(ip -4 route get 1.1.1.1 2>/dev/null | awk '{for(i=1;i<=NF;i++) if($i=="src") {print $(i+1);
exit}}') || true
    fi
    if [[ -z $candidate ]] && command -v hostname >/dev/null; then
        candidate=$(hostname -I 2>/dev/null | awk '{print $1}') || true
    fi
    if valid_ipv4 "$candidate"; then printf '%s' "$candidate"; fi
}
ip_is_local() { ip -o -4 addr show | awk -v wanted="$SERVER_IP" '{split($4,a,"/");if(a[1]==wanted)found=1} END{exit
 !found}'; }
valid_server_ip() {
    valid_ipv4 "$1" && [[ $1 != 127.* && $1 != 0.* ]] && (( ${1%%.*} < 224 )) && [[ ${1##*.} != 0 && ${1##*.} !=
255 ]]
}
choose_server_ip() {
    local iface cidr addr choice suggested='' i addresses=''
    LOCAL_IPS=(); LOCAL_IFACES=(); SERVER_IFACE=''
    if command -v ip >/dev/null; then
        addresses=$(ip -o -4 addr show scope global 2>/dev/null) || warn 'Não foi possível listar as interfaces.
Digite o IP manualmente.'
        while read -r iface cidr; do
            addr=${cidr%/*}
            valid_server_ip "$addr" || continue
            LOCAL_IPS+=("$addr"); LOCAL_IFACES+=("$iface")
        done < <(awk 'NF {print $2,$4}' <<<"$addresses")
    fi
    ui_step 2 'ESCOLHA A INTERFACE DO LABORATÓRIO'
    explain 'Na atividade: Host-Only em 192.168.56.x para as VMs conversarem; NAT para Internet.'
    for i in "${!LOCAL_IPS[@]}"; do
        menu_item "$((i+1))" "${LOCAL_IFACES[i]} — ${LOCAL_IPS[i]}"
        if [[ ${LOCAL_IPS[i]} == 192.168.56.* && -z $suggested ]]; then suggested=$((i+1)); fi
    done
```

```
    [[ -n $suggested ]] || suggested=${LOCAL_IPS[0]:-$(detect_ip)}
    [[ -n $suggested || $MODE != demo ]] || suggested=192.168.56.10
    while :; do
        ask 'Escolha o número da interface OU digite o IPv4 desta VM' "$suggested"
        choice=$REPLY
        if [[ $choice =~ ^[0-9]{1,3}$ ]] && (( 10#$choice >= 1 && 10#$choice <= ${#LOCAL_IPS[@]} )); then
            i=$((10#$choice-1)); SERVER_IP=${LOCAL_IPS[i]}; SERVER_IFACE=${LOCAL_IFACES[i]}
        else SERVER_IP=$choice; fi
        if valid_server_ip "$SERVER_IP"; then break; fi
        warn 'Informe o IP da interface, como 192.168.56.10; não use endereço de rede, broadcast ou loopback.'
    done
    if [[ -z $SERVER_IFACE ]]; then
        for i in "${!LOCAL_IPS[@]}"; do [[ ${LOCAL_IPS[i]} != "$SERVER_IP" ]] || SERVER_IFACE=${LOCAL_IFACES[i]};
done
    fi
    [[ $SERVER_IP != 10.0.2.* ]] || warn 'Esse endereço costuma ser NAT no VirtualBox. Para a atividade entre VMs,
confira o adaptador Host-Only.'
    warn 'O IP deve estar configurado como fixo na VM. O assistente não altera adaptadores do VirtualBox nem a rede
 do sistema.'
}
set_reverse_network() {
    local network=$1 a b c d
    [[ $network == */24 ]] || return 1
    valid_ipv4 "${network%/*}" || return 1
    IFS=. read -r a b c d <<<"${network%/*}"
    [[ $d == 0 ]] || return 1
    REVERSE_NETWORK=$network; REVERSE_PREFIX=$a.$b.$c; REVERSE_ZONE=$c.$b.$a.in-addr.arpa
}
set_zone_paths() {
    if [[ $PLATFORM == debian ]]; then
        if [[ $DNS_ROLE == secondary ]]; then ZONE_DIR=/var/cache/bind/gs-linux
        else ZONE_DIR=/var/lib/bind/gs-linux; fi
    else
        if [[ $DNS_ROLE == secondary ]]; then ZONE_DIR=/var/named/slaves/gs-linux
        else ZONE_DIR=/var/named/gs-linux; fi
    fi
    ZONE_FILE=$ZONE_DIR/$DOMAIN.zone
    REVERSE_FILE=$ZONE_DIR/$REVERSE_ZONE
    if (( LEGACY_STATE )); then ZONE_FILE=$ZONE_DIR/db.zone; REVERSE_FILE=''; fi
}
build_ptr_records() {
    local i j ip name
    PTR_IP=(); PTR_NAME=()
    [[ -n $REVERSE_PREFIX ]] || return 0
    for i in "${!REC_NAME[@]}"; do
        [[ ${REC_TYPE[i]} == A ]] || continue
        ip=${REC_VALUE[i]}
        [[ ${ip%.*} == "$REVERSE_PREFIX" ]] || continue
        name=${REC_NAME[i]}; if [[ $name == @ ]]; then name=$DOMAIN; else name=$name.$DOMAIN; fi
        # Um nome principal por IP: o primeiro cadastrado, mas ns1 tem prioridade.
        for j in "${!PTR_IP[@]}"; do
            if [[ ${PTR_IP[j]} == "$ip" ]]; then
                if [[ ${REC_NAME[i]} == ns1 ]]; then PTR_NAME[j]=$name.; fi
                continue 2
            fi
        done
        PTR_IP+=("$ip"); PTR_NAME+=("$name.")
    done
}

add_record() {
    local name=$1 type=$2 value=$3 priority=${4:-} i
    if [[ $name == ns1 && ( $type != A || $value != "${PRIMARY_IP:-$SERVER_IP}" ) ]]; then
        warn 'ns1 deve continuar apontando para o IP do primário.'; return 1
    fi
    for i in "${!REC_NAME[@]}"; do
        [[ ${REC_NAME[i]} == "$name" ]] || continue
        if [[ ${REC_TYPE[i]} == "$type" && ${REC_VALUE[i]} == "$value" && ${REC_PRIORITY[i]} == "$priority" ]];
then
            warn 'Esse registro já está na lista.'; return 1
        fi
        if [[ $type == CNAME || ${REC_TYPE[i]} == CNAME ]]; then
            warn 'CNAME precisa ser o único registro desse nome. Remova o registro anterior primeiro.'; return 1
        fi
    done
    if [[ $type == CNAME && ( $name == @ || $name == ns1 ) ]]; then
        warn 'O domínio principal e ns1 são reservados para a configuração básica.'; return 1
    fi
    # Um alias só aponta para um nome que já possui A: impede ciclos e aliases quebrados.
    if [[ $type == CNAME || $type == MX ]]; then
        local found=0 target
        for i in "${!REC_NAME[@]}"; do
            target=${REC_NAME[i]}; [[ $target == @ ]] && target=$DOMAIN || target=$target.$DOMAIN
            [[ ${REC_TYPE[i]} == A && $target == "$value" ]] && found=1
        done
        if (( ! found )); then
```

```
            warn 'Cadastre primeiro um registro A para esse destino dentro do seu domínio.'; return 1
        fi
    fi
    REC_NAME+=("$name"); REC_TYPE+=("$type"); REC_VALUE+=("$value"); REC_PRIORITY+=("$priority")
}
base_records() {
    PRIMARY_IP=${PRIMARY_IP:-$SERVER_IP}; WEB_IP=${WEB_IP:-${SECONDARY_IP:-$SERVER_IP}}
    REC_NAME=(@ ns1 www intranet); REC_TYPE=(A A A CNAME)
    REC_VALUE=("$PRIMARY_IP" "$PRIMARY_IP" "$WEB_IP" "www.$DOMAIN"); REC_PRIORITY=('' '' '' '')
}
show_records() {
    local i type_color
    ui_group 'REGISTROS CADASTRADOS'
    if (( UI_WIDTH >= 62 )); then
        printf '  %s%-4s %-22s %-7s %s%s\n' "$MUTED" 'Nº' 'NOME' 'TIPO' 'VALOR' "$RESET"
        ui_rule
    fi
    for i in "${!REC_NAME[@]}"; do
        case ${REC_TYPE[i]} in A) type_color=$CYAN ;; CNAME) type_color=$BLUE ;; MX) type_color=$MAGENTA ;; *)
type_color=$YELLOW ;; esac
        if (( UI_WIDTH >= 62 )); then
            printf '  %s%-4s%s %-22s %s%-7s%s %s%s\n' "$MUTED" "$((i+1))" "$RESET" "${REC_NAME[i]}"
"$type_color$BOLD" "${REC_TYPE[i]}" "$RESET" "${REC_PRIORITY[i]:+${REC_PRIORITY[i]} }" "${REC_VALUE[i]}"
        else
            printf '  %s[ %s ]%s %s  %s%s%s\n' "$MUTED" "$((i+1))" "$RESET" "${REC_NAME[i]}" "$type_color$BOLD"
"${REC_TYPE[i]}" "$RESET"
            printf '        %s%s\n' "${REC_PRIORITY[i]:+${REC_PRIORITY[i]} }" "${REC_VALUE[i]}"
        fi
    done
    ui_rule
    info 'Você pode editar valores ou remover registros. ns1 acompanha o IP do primário e é necessário ao SOA/NS.'
}
prompt_record() {
    local type=$1 name=${2:-} value priority='' default_value=${3:-}
    while [[ -z $name ]]; do
        ask 'Nome do registro (ex.: loja, mail ou @ para o domínio principal)'
        name=${REPLY,,}; name=${name%.}
        [[ $name == "$DOMAIN" ]] && name=@
        [[ $name == *."$DOMAIN" ]] && name=${name%."$DOMAIN"}
        if valid_owner "$name" "$type"; then break; fi
        warn 'Nome inválido. Use letras sem acento, números e hífens; TXT também aceita _.'
        name=''
    done
    case $type in
        A)
            while :; do
                ask 'IPv4 de destino (ex.: 192.168.56.20)' "${default_value:-$SERVER_IP}"; value=$REPLY
                if valid_ipv4 "$value"; then break; fi
                warn 'IPv4 inválido. Cada parte deve ir de 0 a 255, sem zeros à esquerda.'
            done ;;
        CNAME|MX)
            info "O destino precisa ter um registro A nesta zona. Ex.: www.$DOMAIN."
            [[ $type == MX ]] && warn 'MX só publica o endereço: não instala um servidor de e-mail.'
            while :; do
                ask 'Nome completo de destino' "${default_value:-$DOMAIN}"; value=${REPLY,,}; value=${value%.}
                if valid_domain "$value"; then break; fi
                warn "Use um nome completo, como www.$DOMAIN, nunca um IP."
            done
            if [[ $type == MX ]]; then
                while :; do
                    ask 'Prioridade do MX (menor número = maior preferência)' 10; priority=$REPLY
                    if valid_priority "$priority"; then priority=$((10#$priority)); break; fi
                    warn 'Digite um número entre 0 e 65535.'
                done
            fi ;;
        TXT)
            while :; do
                ask 'Texto (até 200 caracteres; não precisa colocar aspas)' "$default_value"; value=$REPLY
                if LC_ALL=C valid_txt "$value"; then break; fi
                warn 'Use de 1 a 200 caracteres visíveis, sem acentos ou quebras de linha.'
            done ;;
    esac
    if add_record "$name" "$type" "$value" "$priority"; then ok 'Registro adicionado.'; else return 1; fi
}
remove_record() {
    local index i target
    show_records
    ask 'Número do registro a remover (0 = voltar)' 0
    [[ $REPLY =~ ^[0-9]{1,5}$ ]] || { warn 'Digite um número da lista.'; return; }
    index=$((10#$REPLY - 1))
    (( index == -1 )) && return 0
    if (( index < 0 || index >= ${#REC_NAME[@]} )); then warn 'Escolha um número da lista.'; return; fi
    if [[ ${REC_NAME[index]} == ns1 ]]; then warn 'ns1 identifica o primário no SOA/NS. Para trocar seu IP, refaça
o assistente.'; return; fi
    target=${REC_NAME[index]}; [[ $target == @ ]] && target=$DOMAIN || target=$target.$DOMAIN
    for i in "${!REC_NAME[@]}"; do
```

```
        if [[ ( ${REC_TYPE[i]} == CNAME || ${REC_TYPE[i]} == MX ) && ${REC_VALUE[i]} == "$target" ]]; then
            warn 'Outro registro depende desse destino. Remova o alias/MX primeiro.'; return
        fi
    done
    unset 'REC_NAME[index]' 'REC_TYPE[index]' 'REC_VALUE[index]' 'REC_PRIORITY[index]'
    REC_NAME=("${REC_NAME[@]}"); REC_TYPE=("${REC_TYPE[@]}"); REC_VALUE=("${REC_VALUE[@]}");
REC_PRIORITY=("${REC_PRIORITY[@]}")
    ok 'Registro removido.'
}
edit_record() {
    local index name type value
    local -a old_names=("${REC_NAME[@]}") old_types=("${REC_TYPE[@]}") old_values=("${REC_VALUE[@]}")
old_priorities=("${REC_PRIORITY[@]}")
    show_records
    ask 'Número do registro cujo valor deseja editar (0 = voltar)' 0
    [[ $REPLY =~ ^[0-9]{1,5}$ ]] || { warn 'Digite um número da lista.'; return; }
    index=$((10#$REPLY-1))
    (( index == -1 )) && return 0
    if (( index < 0 || index >= ${#REC_NAME[@]} )); then warn 'Escolha um número da lista.'; return; fi
    name=${REC_NAME[index]}; type=${REC_TYPE[index]}; value=${REC_VALUE[index]}
    if [[ $name == ns1 ]]; then warn 'Para trocar o IP de ns1, refaça o assistente com o IP correto do primário.';
return; fi
    unset 'REC_NAME[index]' 'REC_TYPE[index]' 'REC_VALUE[index]' 'REC_PRIORITY[index]'
    REC_NAME=("${REC_NAME[@]}"); REC_TYPE=("${REC_TYPE[@]}"); REC_VALUE=("${REC_VALUE[@]}");
REC_PRIORITY=("${REC_PRIORITY[@]}")
    if ! prompt_record "$type" "$name" "$value"; then
        REC_NAME=("${old_names[@]}"); REC_TYPE=("${old_types[@]}"); REC_VALUE=("${old_values[@]}");
REC_PRIORITY=("${old_priorities[@]}")
        warn 'Edição inválida. O registro anterior foi preservado.'
    fi
}
# Lê os registros FINAIS (depois de editar/remover) e avisa quando www não aponta para esta VM.
www_notice() {
    local i found=0 apex=0
    for i in "${!REC_NAME[@]}"; do
        if [[ ${REC_NAME[i]} == @ && ${REC_TYPE[i]} == A ]]; then apex=1; fi
        [[ ${REC_NAME[i]} == www ]] || continue
        found=1
        [[ ${REC_TYPE[i]} == A && ${REC_VALUE[i]} != "$SERVER_IP" ]] || continue
        if [[ -n $SECONDARY_IP && ${REC_VALUE[i]} == "$SECONDARY_IP" ]]; then
            warn "www.$DOMAIN aponta para o secundário (${REC_VALUE[i]}). Execute este assistente também na VM do
secundário; sem isso o site e o DNS dele não respondem."
        else
            warn "www.$DOMAIN aponta para ${REC_VALUE[i]}, que não é esta VM ($SERVER_IP). O Apache dessa máquina
precisa estar instalado e configurado."
        fi
    done
    if (( ! found )); then
        if (( apex )); then
            info "Não há registro www: www.$DOMAIN não será resolvido pelo DNS. O site continua acessível pelo IP e
 por $DOMAIN."
        else
            info "Não há registro www nem registro A para $DOMAIN: nenhum desses nomes será resolvido pelo DNS. O
site continua acessível pelo IP desta VM."
        fi
    fi
}

wizard() {
    local suggested choice i
    while :; do
        LEGACY_STATE=0; PRIMARY_IP=''; SECONDARY_IP=''; WEB_IP=''
        ui_step 1 'DOMÍNIO E PAPEL DESTA VM'
        while :; do
            ask 'Domínio da atividade (seu nome + .fiap, sem http://)' nome.fiap
            DOMAIN=${REPLY,,}; DOMAIN=${DOMAIN%.}
            if valid_domain "$DOMAIN" && [[ $DOMAIN != *.local ]]; then break; fi
            warn 'Exemplo válido: aluno.fiap. Evite .local, reservado para descoberta de rede.'
        done
        ui_group 'COMO ESTA VM PARTICIPA DO LABORATÓRIO'
        while :; do
            menu_item 1 'Primário: criar e editar as zonas (primeira VM)'
            menu_item 2 'Secundário: receber as zonas do primário (segunda VM)'
            ask 'Papel DESTA VM' 1
            case $REPLY in 1) DNS_ROLE=primary; break ;; 2) DNS_ROLE=secondary; break ;; *) warn 'Escolha 1 ou 2.'
;; esac
        done
        choose_server_ip
        if [[ $DNS_ROLE == primary ]]; then
            PRIMARY_IP=$SERVER_IP
            suggested=${SERVER_IP%.*}.20
            [[ $suggested != "$SERVER_IP" ]] || suggested=${SERVER_IP%.*}.21
            while :; do
                ask 'IP reservado ao secundário (pode ser criado depois; 0 = usar só esta VM)' "$suggested"
                [[ $REPLY != 0 ]] || break
                if valid_server_ip "$REPLY" && [[ $REPLY != "$SERVER_IP" ]]; then SECONDARY_IP=$REPLY; break; fi
```

```
                warn 'Informe outro IP válido para a segunda VM, ou 0.'
            done
            WEB_IP=${SECONDARY_IP:-$SERVER_IP}
            base_records
        else
            SECONDARY_IP=$SERVER_IP
            while :; do
                ask 'IP do servidor PRIMÁRIO que já foi configurado' "${SERVER_IP%.*}.10"
                if valid_server_ip "$REPLY" && [[ $REPLY != "$SERVER_IP" ]]; then PRIMARY_IP=$REPLY; break; fi
                warn 'O primário precisa ter outro IP válido.'
            done
            WEB_IP=$SERVER_IP
            REC_NAME=(); REC_TYPE=(); REC_VALUE=(); REC_PRIORITY=()
            explain 'No secundário, os registros vêm da transferência. Edite-os sempre no primário.'
        fi
        ui_step 3 'ZONA REVERSA'
        while :; do
            ask 'Rede /24 da zona reversa (use a mesma nas duas VMs)' "${PRIMARY_IP%.*}.0/24"
            if set_reverse_network "$REPLY"; then break; fi
            warn 'Este perfil atende redes /24, como 192.168.56.0/24. Informe o endereço terminado em .0/24.'
        done
        set_zone_paths
        ui_step 4 'REGISTROS DNS'
        while [[ $DNS_ROLE == primary ]]; do
            show_records
            ui_group 'ADICIONAR UM REGISTRO'
            menu_item 1 'A — nome → IPv4'
            menu_item 2 'CNAME — apelido → outro nome'
            menu_item 3 'MX — servidor de e-mail'
            menu_item 4 'TXT — texto'
            ui_group 'REVISAR OU ENTENDER'
            menu_item 5 'Remover registro'
            menu_item 6 'Não entendi os registros: abrir explicação'
            menu_item 7 'Editar o valor de um registro existente'
            printf '\n'
            menu_item 0 'Concluir registros'
            ask 'Escolha uma opção' 0
            case $REPLY in
                1) prompt_record A || true ;; 2) prompt_record CNAME || true ;; 3) prompt_record MX || true ;; 4)
prompt_record TXT || true ;;
                5) remove_record ;; 6) lesson records; pause_reading ;; 7) edit_record ;; 0) break ;; *) warn
'Escolha uma opção de 0 a 7.' ;;
            esac
        done
        ui_step 5 'RESUMO ANTES DE CONTINUAR'
        ui_detail 'Domínio' "$DOMAIN"
        ui_detail 'IP desta VM' "$SERVER_IP"
        ui_detail 'Sistema' "$PLATFORM"
        if [[ $DNS_ROLE == primary ]]; then ui_detail 'Papel' 'Primário (master)'
        else ui_detail 'Papel' 'Secundário (slave)'; fi
        ui_detail 'Primário' "$PRIMARY_IP"
        ui_detail 'Secundário' "${SECONDARY_IP:-nenhum}"
        ui_detail 'Zona reversa' "$REVERSE_ZONE"
        ui_detail 'SOA' '28800 / 14400 / 86400 / 600'
        ui_detail 'Serial' 'AAAAMMDDnn'
        if [[ $DNS_ROLE == primary ]]; then
            show_records
            info 'Um PTR por IP dentro da rede /24 escolhida (ns1 tem prioridade no IP do primário); CNAME não
recebe PTR.'
            www_notice
            for i in "${!REC_NAME[@]}"; do
                if [[ ${REC_TYPE[i]} == A && ${REC_VALUE[i]%.*} != "$REVERSE_PREFIX" ]]; then
                    warn "${REC_NAME[i]} (${REC_VALUE[i]}) está fora de $REVERSE_NETWORK e não receberá PTR nesta
zona."
                fi
            done
        else info 'As zonas direta e reversa serão recebidas do primário. Ele deve autorizar o IP desta VM.'; fi
        ui_group 'O QUE SERÁ APLICADO'
        info 'Instalar: BIND e Apache; template Bare / Start Bootstrap (MIT), conferido por SHA-256. Se o download
falhar, usa a cópia local em template/.'
        warn 'A configuração principal do BIND será substituída; o Apache servirá este site como padrão.'
        info 'Arquivos alterados terão backup. Pacotes instalados permanecem se houver falha.'
        [[ $PLATFORM != debian ]] || info 'APT: fontes de CD/DVD serão desativadas; se faltar fonte online,
adicionar repositórios oficiais com backup.'
        info 'Firewall ativo: liberar somente DNS (TCP/UDP 53) e HTTP (TCP 80). SELinux permanece ativo.'
        ui_rule
        ask 'S = continuar, R = refazer as respostas, N = sair' N; choice=${REPLY,,}
        case $choice in s|sim) return ;; r) continue ;; *) info 'Cancelado. Nenhuma configuração aplicada.'; exit 0
 ;; esac
    done
}

preflight() {
    (( EUID == 0 )) || die 'Para instalar: sudo bash gs-linux.sh. Para treinar: bash gs-linux.sh --demo.'
    command -v systemctl >/dev/null || die 'Esta instalação precisa de systemd (use uma VM completa).'
    [[ -d /run/systemd/system ]] || die 'O systemd não está ativo. Use uma VM Debian/Rocky, não um contêiner
```

```
comum.'
    command -v flock >/dev/null || die 'Falta flock (pacote util-linux).'
    exec 9>/run/lock/gs-linux.lock
    flock -n 9 || die 'Já existe outra execução deste instalador.'
    [[ ! -L $STATE_DIR && ! -L $WEB_ROOT ]] || die 'Diretório de trabalho é um link simbólico; interrompendo para
preservar os arquivos.'
}
run() {
    local label=$1; shift
    info "$label"
    if "$@" >>"$LOG" 2>&1; then ok "$label"; else
        tail -n 15 "$LOG" >&2
        die "Falha: $label. Consulte $LOG. Não prossiga ignorando esse erro."
    fi
}
# Desativa apenas mídia removível. Não muda mirrors, componentes, chaves ou opções
# das demais fontes. A transformação é preparada em arquivo temporário primeiro.
filter_apt_media() {
    local source_file=$1
    if [[ $source_file != *.sources ]]; then
        sed -E '/^[[:space:]]*deb(-src)?[[:space:]]+(\[[^]]*\][[:space:]]+)?cdrom:/s/^/# GS Linux: mídia
desativada: /' "$source_file"
        return
    fi
    awk '
        function flush(
i,key,value,field,uris,disabled,first,last,enabled,nparts,part,level,ch,token,kept,removed) {
            uris=""; disabled=0; first=0; last=0; enabled=0; field=""
            for(i=1;i<=n;i++) {
                if(lines[i] ~ /^#/) continue
                if(lines[i] ~ /^[^ \t][^:]*:/) {
                    key=lines[i]; sub(/:.*/,"",key); field=tolower(key)
                    value=lines[i]; sub(/^[^:]*:[ \t]*/,"",value)
                    if(field=="enabled") { enabled=i; if(tolower(value) ~ /^no[ \t]*$/) disabled=1 }
                    if(field=="uris") { first=i; last=i; uris=value }
                } else if(field=="uris" && lines[i] ~ /^[ \t]/) {
                    value=lines[i]; sub(/^[ \t]+/,"",value); uris=uris " " value; last=i
                }
            }
            kept=""; removed=0; level=0; token=""
            # Nomes de discos podem conter espaços dentro de colchetes.
            for(i=1;i<=length(uris)+1;i++) {
                ch=(i<=length(uris) ? substr(uris,i,1) : " ")
                if(ch=="[") level++
                if(ch=="]" && level>0) level--
                if(ch ~ /[ \t]/ && level==0) {
                    if(token ~ /^cdrom:/) removed=1
                    else if(token!="") kept=kept (kept!="" ? " " : "") token
                    token=""
                } else token=token ch
            }
            if(disabled || !removed) { for(i=1;i<=n;i++) print lines[i] }
            else if(kept=="") {
                for(i=1;i<=n;i++) { if(i==enabled) print "Enabled: no"; else print lines[i] }
                if(!enabled) print "Enabled: no"
            } else {
                for(i=1;i<=n;i++) {
                    if(i==first) print "URIs: " kept
                    else if(i<first || i>last) print lines[i]
                }
            }
            delete lines; n=0
        }
        /^[ \t]*$/ { flush(); print; next }
        { lines[++n]=$0 }
        END { if(n) flush() }
    ' "$source_file"
}
restore_apt_sources() {
    local i failed=0
    for i in "${!APT_PATHS[@]}"; do
        if [[ ${APT_EXISTS[i]} == 1 ]]; then
            cp -a -- "$APT_BACKUP/$i" "${APT_PATHS[i]}" || failed=1
        else rm -f -- "${APT_PATHS[i]}" || failed=1; fi
    done
    APT_REPAIR_PENDING=0
    if (( failed )); then warn "Falha ao restaurar fontes APT. Consulte o backup: $APT_BACKUP"; return 1; fi
    warn "Fontes APT anteriores restauradas. Backup: $APT_BACKUP"
}
prepare_debian_sources() {
    # Argumentos permitem testar tudo em pastas temporárias, sem tocar em /etc.
    local apt_dir=${1:-/etc/apt} backups=${2:-/var/backups/gs-linux} codename=${3:-$DEBIAN_CODENAME}
    local keyring=${4:-/usr/share/keyrings/debian-archive-keyring.gpg}
    local stage file relative destination i uri_file
    local -a originals=() staged=()
    case $codename in bullseye|bookworm|trixie) ;; *) die 'Não foi possível determinar a versão Debian para
```

```
configurar o APT.' ;; esac
    APT_PATHS=(); APT_EXISTS=(); APT_REPAIR_PENDING=0
    stage=$(mktemp -d "$WORK/apt-sources.XXXXXXXX")
    mkdir "$stage/sources.list.d"
    : >"$stage/sources.list"
    for file in "$apt_dir/sources.list" "$apt_dir/sources.list.d/"*.list "$apt_dir/sources.list.d/"*.sources; do
        [[ -e $file ]] || continue
        [[ -f $file && -r $file ]] || die "Fonte APT não é um arquivo legível: $file"
        relative=${file#"$apt_dir/"}
        filter_apt_media "$file" >"$stage/$relative"
        if ! cmp -s "$file" "$stage/$relative"; then
            [[ ! -L $file ]] || die "A fonte de mídia é um link simbólico: $file. Arquivo preservado."
            originals+=("$file"); staged+=("$stage/$relative")
        fi
    done
    # O próprio APT interpreta os formatos/Enabled/Types. --print-uris não baixa nada.
    # Esta saída pode conter endereços de mirrors privados: nunca a imprima.
    uri_file=$stage/targets.txt
    if ! apt-get -o "Dir::Etc::sourcelist=$stage/sources.list" -o "Dir::Etc::sourceparts=$stage/sources.list.d"
--print-uris update >"$uri_file" 2>&1; then
        die 'O APT não conseguiu interpretar as fontes preparadas. Nenhum arquivo de repositório foi alterado.'
    fi
    if ! grep -Eq "^'https?://.*(/binary-[^/]+/Packages|/Packages)[^ ]*' " "$uri_file"; then
        [[ -s $keyring ]] || die 'Falta o chaveiro oficial Debian (debian-archive-keyring). Não é seguro adicionar
fontes sem ele.'
        destination=$apt_dir/sources.list.d/gs-linux-debian.sources
        [[ ! -e $destination && ! -L $destination ]] || die "Já existe $destination, mas não fornece pacotes
online. Arquivo preservado; confira o conteúdo."
        cat >"$stage/sources.list.d/gs-linux-debian.sources" <<EOF
# GS Linux: fontes oficiais para uma instalação sem mirror online.
# APT verifica a assinatura Debian e os hashes dos pacotes; não usa trusted=yes.
Types: deb
URIs: http://deb.debian.org/debian
Suites: $codename $codename-updates
Components: main
Signed-By: $keyring

Types: deb
URIs: http://security.debian.org/debian-security
Suites: $codename-security
Components: main
Signed-By: $keyring
EOF
        originals+=("$destination"); staged+=("$stage/sources.list.d/gs-linux-debian.sources")
        info "Não há fonte de pacotes pela Internet. Preparando repositórios oficiais Debian $codename."
    fi
    if (( ${#originals[@]} == 0 )); then info 'Fontes APT já possuem acesso online e não exigem reparo de CD/DVD.';
 return; fi
    install -d -m 700 "$backups"
    APT_BACKUP=$(mktemp -d "$backups/apt-$(date +%Y%m%d-%H%M%S).XXXXXXXX")
    for i in "${!originals[@]}"; do
        file=${originals[i]}
        APT_PATHS+=("$file")
        if [[ -e $file ]]; then
            cp -a -- "$file" "$APT_BACKUP/$i"
            APT_EXISTS+=(1)
        else APT_EXISTS+=(0); fi
        printf '%s\t%s\t%s\n' "$i" "${APT_EXISTS[i]}" "$file" >>"$APT_BACKUP/manifest.txt"
    done
    APT_REPAIR_PENDING=1
    info "Preparando APT para usar a Internet. Backup das fontes: $APT_BACKUP"
    install -d "$apt_dir/sources.list.d"
    for i in "${!originals[@]}"; do
        file=${originals[i]}
        if [[ ${APT_EXISTS[i]} == 1 ]]; then cat "${staged[i]}" >"$file"
        else install -m 644 "${staged[i]}" "$file"; fi
    done
}
install_packages() {
    if [[ $PLATFORM == debian ]]; then
        prepare_debian_sources /etc/apt /var/backups/gs-linux "$DEBIAN_CODENAME"
        info 'Atualizando a lista de pacotes (todas as fontes precisam responder corretamente)'
        if apt-get -o APT::Update::Error-Mode=any -o Acquire::Retries=2 -o Acquire::http::Timeout=30 -o
Acquire::https::Timeout=30 update >>"$LOG" 2>&1; then
            APT_REPAIR_PENDING=0
            ok 'Lista de pacotes atualizada e assinaturas verificadas pelo APT.'
        else
            if (( APT_REPAIR_PENDING )); then restore_apt_sources || true; fi
            tail -n 15 "$LOG" >&2
            die "Falha ao atualizar os pacotes. Confira Internet, horário da VM e fontes APT. Log: $LOG"
        fi
        run 'Instalando BIND, Apache e ferramentas de teste' env DEBIAN_FRONTEND=noninteractive apt-get -o
DPkg::Lock::Timeout=60 install -y bind9 bind9-utils dnsutils apache2 curl ca-certificates iproute2
    else
        local -a packages=(bind bind-utils httpd ca-certificates iproute policycoreutils)
        # Rocky pode trazer curl-minimal: ele já atende ao script; não o troque à força.
```

```
        if ! command -v curl >/dev/null; then packages+=(curl); fi
        run 'Instalando BIND, Apache e ferramentas de teste' dnf -y --setopt=timeout=30 --setopt=retries=2 install
"${packages[@]}"
    fi
}
check_ports() {
    local line
    while IFS= read -r line; do
        [[ -n $line ]] || continue
        # systemd-resolved pode manter seu stub 127.0.0.53/54: não colide com os IPs configurados.
        [[ $line == *127.0.0.53:53* || $line == *127.0.0.54:53* ]] && continue
        if [[ $line != *'"named"'* && $line != *'"apache2"'* && $line != *'"httpd"'* ]]; then
            warn "$line"
            die 'Outro programa está usando a porta 53 ou 80. Resolva o conflito antes de instalar.'
        fi
    done < <(ss -H -lntup '( sport = :53 or sport = :80 )')
}
backup_path() {
    local path=$1
    [[ ! -L $path || ${2:-} == link ]] || die "Arquivo gerenciado é um link simbólico: $path. Revise-o antes de
continuar."
    SAVED_PATHS+=("$path")
    if [[ -e $path || -L $path ]]; then
        SAVED_EXISTS+=(1)
        mkdir -p -- "$BACKUP/files$(dirname "$path")"
        cp -a -- "$path" "$BACKUP/files$path"
        printf 'EXISTIA %s\n' "$path" >>"$BACKUP/manifest.txt"
    else SAVED_EXISTS+=(0); printf 'NOVO %s\n' "$path" >>"$BACKUP/manifest.txt"; fi
}
begin_transaction() {
    if [[ ! -f $STATE_DIR/state ]]; then
        local path
        for path in "$WEB_ROOT" "$WEB_CONF" "$(dirname "$ZONE_FILE")"; do
            [[ ! -e $path ]] || die "Já existe $path sem identificação do assistente. Preservado: mova-o ou use uma
 VM limpa."
        done
    fi
    BACKUP=/var/backups/gs-linux/$(date +%Y%m%d-%H%M%S)-$$
    install -d -m 700 "$BACKUP"
    systemctl is-active --quiet "$DNS_SERVICE" && DNS_WAS_ACTIVE=1
    systemctl is-active --quiet "$WEB_SERVICE" && WEB_WAS_ACTIVE=1
    systemctl is-enabled --quiet "$DNS_SERVICE" && DNS_WAS_ENABLED=1
    systemctl is-enabled --quiet "$WEB_SERVICE" && WEB_WAS_ENABLED=1
    # Só começamos a alterar arquivos depois de concluir todos os backups.
    backup_path "$DNS_CONF"; backup_path "$ZONE_DIR"; backup_path "$WEB_CONF"
    [[ -z $WEB_LINK ]] || backup_path "$WEB_LINK" link
    backup_path "$WEB_ROOT"; backup_path "$STATE_DIR/state"
    TRANSACTION=1
}
rollback() {
    local i path item kind zone service status=0
    warn 'Falha detectada. Restaurando os arquivos e as regras alteradas nesta execução...'
    for ((i=${#SAVED_PATHS[@]}-1;i>=0;i--)); do
        path=${SAVED_PATHS[i]}
        rm -rf -- "$path" || status=1
        if [[ ${SAVED_EXISTS[i]} == 1 ]]; then cp -a -- "$BACKUP/files$path" "$path" || status=1; fi
    done
    for item in "${FW_ADDED[@]}"; do
        IFS='|' read -r kind zone service <<< "$item"
        case $kind in
            runtime) firewall-cmd --zone="$zone" --remove-service="$service" || status=1 ;;
            permanent) firewall-cmd --permanent --zone="$zone" --remove-service="$service" || status=1 ;;
            ufw) ufw --force delete allow to "$zone" port "${service%/*}" proto "${service#*/}" || status=1 ;;
        esac
    done
    if command -v restorecon >/dev/null; then restorecon -R "$DNS_CONF" "$(dirname "$ZONE_FILE")" "$WEB_ROOT"
"$WEB_CONF" 2>/dev/null || true; fi
    if (( DNS_WAS_ACTIVE )); then systemctl restart "$DNS_SERVICE" || status=1; else systemctl stop "$DNS_SERVICE"
|| status=1; fi
    if (( WEB_WAS_ACTIVE )); then systemctl restart "$WEB_SERVICE" || status=1; else systemctl stop "$WEB_SERVICE"
|| status=1; fi
    if (( ! DNS_WAS_ENABLED )); then systemctl disable "$DNS_SERVICE" || status=1; fi
    if (( ! WEB_WAS_ENABLED )); then systemctl disable "$WEB_SERVICE" || status=1; fi
    if (( status )); then warn "A restauração teve falhas: consulte $LOG e os backups em $BACKUP."; else ok
'Arquivos anteriores restaurados. Pacotes instalados foram mantidos.'; fi
}
cleanup() {
    local result=$?
    trap - EXIT ERR INT TERM
    set +e
    if (( APT_REPAIR_PENDING )); then restore_apt_sources; fi
    if (( result != 0 && TRANSACTION )); then rollback >>"$LOG" 2>&1; warn "Instalação interrompida; restauração
registrada em $LOG. Backup: $BACKUP"; fi
    [[ -z $WORK ]] || rm -rf -- "$WORK"
    if (( result != 0 )) && [[ ${EXAM_NETWORK_PENDING:-0} == 1 ]]; then exam_restore_network; fi
    exit "$result"
```

```
}

# Template fixado em um commit: atualizações do autor não mudam o conteúdo silenciosamente.
# Baixa o template sem encerrar o script: quem decide o que fazer com a falha é download_template.
fetch_template() {
    local base=$1 file out=${LOG:-/dev/null}
    local -a opts=(--fail --location --silent --show-error --proto '=https' --proto-redir '=https' --connect-
timeout 15 --max-time 90 --retry 2)
    for file in index.html css/styles.css js/scripts.js assets/favicon.ico; do
        curl "${opts[@]}" "$base/dist/$file" -o "$WORK/site/$file" >>"$out" 2>&1 || return 1
    done
    curl "${opts[@]}" "$base/LICENSE" -o "$WORK/site/LICENSE" >>"$out" 2>&1 || return 1
    curl "${opts[@]}" 'https://cdn.jsdelivr.net/npm/bootstrap@5.2.3/dist/js/bootstrap.bundle.min.js' -o
"$WORK/site/js/bootstrap.bundle.min.js" >>"$out" 2>&1 || return 1
}
# Confere os SHA-256 conhecidos. Arquivo ausente, alterado ou ilegível = falha.
verify_template() {
    local dir=$1
    [[ -d $dir ]] || return 1
    ( cd -- "$dir" && printf '%s\n' "${TEMPLATE_SHA256[@]}" | sha256sum --check --strict --quiet - )
>>"${LOG:-/dev/null}" 2>&1
}
# Copia SOMENTE os arquivos listados (e já verificados): nada extra da pasta entra no site.
use_local_template() {
    local entry file
    verify_template "$TEMPLATE_DIR" || return 1
    for entry in "${TEMPLATE_SHA256[@]}"; do
        file=${entry#*  }
        install -D -m 644 "$TEMPLATE_DIR/$file" "$WORK/site/$file" || return 1
    done
}
download_template() {
    local base=https://raw.githubusercontent.com/StartBootstrap/startbootstrap-
bare/a9b948d6138fd3fbfaa6a26908ffbd47c50f9881
    install -d "$WORK/site/css" "$WORK/site/js" "$WORK/site/assets"
    info 'Baixando o template (Bare / Start Bootstrap, versão fixada) e conferindo os SHA-256'
    if fetch_template "$base" && verify_template "$WORK/site"; then
        TEMPLATE_SOURCE=internet
        ok 'Template baixado e verificado por SHA-256.'
    else
        warn 'Download do template falhou ou não bate com os SHA-256 conhecidos. Tentando a cópia local.'
        rm -rf -- "$WORK/site"; install -d "$WORK/site/css" "$WORK/site/js" "$WORK/site/assets"
        if use_local_template; then
            TEMPLATE_SOURCE=local
            ok "USANDO A CÓPIA LOCAL do template, verificada por SHA-256: $TEMPLATE_DIR"
        else
            die "Sem template válido: o download falhou e a cópia local ($TEMPLATE_DIR) está ausente ou alterada.
Mantenha a pasta template/ ao lado do script."
        fi
    fi
    grep -qi '<!doctype html>' "$WORK/site/index.html" || die 'O arquivo do template não parece ser HTML.'
    sed -i
"s|https://cdn.jsdelivr.net/npm/bootstrap@5.2.3/dist/js/bootstrap.bundle.min.js|js/bootstrap.bundle.min.js|;s|A
Bootstrap 5 Starter Template|GS Linux: $DOMAIN|;s|A complete project boilerplate built with Bootstrap|Servidor DNS
e Apache configurados|" "$WORK/site/index.html"
    printf '%s\n' "$DOMAIN" >"$WORK/site/gs-health.txt"
}
next_zone_serial() {
    local previous=${1:-0} today=${2:-$(date +%Y%m%d)} floor
    [[ $today =~ ^[0-9]{8}$ && $previous =~ ^[0-9]{1,10}$ ]] || die 'Data/serial inválidos.'
    floor=$((10#${today}00))
    previous=$((10#$previous))
    (( floor+99 < 4294967295 && previous < 4294967295 )) || die 'Serial fora do limite de 32 bits do DNS.'
    if (( previous < floor )); then printf '%s' "$floor"
    elif (( previous < floor+99 )); then printf '%s' "$((previous+1))"
    else die 'Serial futuro ou limite de 100 versões no dia atingido. Confira o relógio/serial antes de publicar.';
 fi
}
render_soa() {
    cat <<EOF
; Gerado pelo GS Linux — atividade FIAP.
\$TTL 600
@ IN SOA ns1.$DOMAIN. hostmaster.$DOMAIN. (
    $1 ; serial: AAAAMMDDnn
    28800      ; refresh: 8 horas
    14400      ; retry: 4 horas
    86400      ; expire: 1 dia
    600 )      ; negative cache: 10 minutos
@ IN NS ns1.$DOMAIN.
EOF
}
render_zone() {
    local output=$1 serial=$2 i value
    render_soa "$serial" >"$output"
    for i in "${!REC_NAME[@]}"; do
        value=${REC_VALUE[i]}
```

```
        case ${REC_TYPE[i]} in
            A) printf '%s IN A %s\n' "${REC_NAME[i]}" "$value" ;;
            CNAME) printf '%s IN CNAME %s.\n' "${REC_NAME[i]}" "$value" ;;
            MX) printf '%s IN MX %s %s.\n' "${REC_NAME[i]}" "${REC_PRIORITY[i]}" "$value" ;;
            TXT) value=${value//\\/\\\\}; value=${value//\"/\\\"}; printf '%s IN TXT "%s"\n' "${REC_NAME[i]}"
"$value" ;;
        esac
    done >>"$output"
}
render_reverse() {
    local output=$1 serial=$2 i
    render_soa "$serial" >"$output"
    build_ptr_records
    for i in "${!PTR_IP[@]}"; do
        printf '%s IN PTR %s\n' "${PTR_IP[i]##*.}" "${PTR_NAME[i]}"
    done >>"$output"
}
render_zone_declaration() {
    local zone=$1 file=$2
    printf 'zone "%s" IN {\n' "$zone"
    if [[ $DNS_ROLE == secondary ]]; then
        printf '    type slave;\n    masters { %s; };\n    file "%s";\n    notify no;\n' "$PRIMARY_IP" "$file"
    else
        printf '    type master;\n    file "%s";\n    allow-update { none; };\n' "$file"
        if [[ -n $SECONDARY_IP ]]; then
            printf '    allow-transfer { %s; };\n    also-notify { %s; };\n    notify yes;\n' "$SECONDARY_IP"
"$SECONDARY_IP"
        else printf '    notify no;\n'; fi
    fi
    printf '};\n'
}
render_named() {
    local output=$1 cache
    if [[ $PLATFORM == debian ]]; then cache=/var/cache/bind; else cache=/var/named; fi
    cat >"$output" <<EOF
// GS Linux: DNS autoritativo. master/slave são os nomes compatíveis com as versões suportadas.
options {
    directory "$cache";
    listen-on port 53 { 127.0.0.1; $SERVER_IP; };
    listen-on-v6 { none; };
    recursion no;
    allow-query { any; };
    allow-transfer { none; };
    transfer-source $SERVER_IP;
    notify-source $SERVER_IP;
    dnssec-validation auto;
};
EOF
    render_zone_declaration "$DOMAIN" "$ZONE_FILE" >>"$output"
    [[ -z $REVERSE_ZONE ]] || render_zone_declaration "$REVERSE_ZONE" "$REVERSE_FILE" >>"$output"
}
render_web() {
    cat >"$1" <<EOF
# GS Linux: arquivo ordenado antes dos outros sites para ser o padrão.
ServerName $DOMAIN
ServerTokens Prod
ServerSignature Off
<VirtualHost *:80>
    ServerName $DOMAIN
    ServerAlias www.$DOMAIN $SERVER_IP
    DocumentRoot "$WEB_ROOT"
    DirectoryIndex index.html
    <Directory "$WEB_ROOT">
        Options -Indexes -Includes -ExecCGI
        AllowOverride None
        Require all granted
    </Directory>
</VirtualHost>
EOF
}
apply_configuration() {
    local serial previous=0 candidate file
    if [[ $DNS_ROLE == primary ]]; then
        for file in "$ZONE_FILE" "$REVERSE_FILE" "$ZONE_DIR/db.zone"; do
            [[ -f $file ]] || continue
            candidate=$(awk '/; serial:/ {print $1; exit}' "$file")
            if [[ $candidate =~ ^[0-9]{1,10}$ ]] && (( 10#$candidate > previous )); then
previous=$((10#$candidate)); fi
        done
        serial=$(next_zone_serial "$previous")
        render_zone "$WORK/db.zone" "$serial"
        render_reverse "$WORK/db.reverse" "$serial"
        run 'Validando sintaxe da zona DNS direta' named-checkzone "$DOMAIN" "$WORK/db.zone"
        run 'Validando sintaxe da zona DNS reversa' named-checkzone "$REVERSE_ZONE" "$WORK/db.reverse"
    fi
    render_named "$WORK/named.conf"
```

```
    render_web "$WORK/web.conf"
    run 'Validando sintaxe da configuração BIND' named-checkconf "$WORK/named.conf"
    if [[ $DNS_ROLE == primary ]]; then
        install -d -m 750 -o root -g "$DNS_GROUP" "$ZONE_DIR"
        install -m 640 -o root -g "$DNS_GROUP" "$WORK/db.zone" "$ZONE_FILE"
        install -m 640 -o root -g "$DNS_GROUP" "$WORK/db.reverse" "$REVERSE_FILE"
    else
        install -d -m 750 -o "$DNS_GROUP" -g "$DNS_GROUP" "$ZONE_DIR"
        # O BIND receberá cópias novas; nunca geramos registros locais no secundário.
        rm -f -- "$ZONE_FILE" "$ZONE_FILE.jnl" "$REVERSE_FILE" "$REVERSE_FILE.jnl"
    fi
    install -m 640 -o root -g "$DNS_GROUP" "$WORK/named.conf" "$DNS_CONF"
    install -m 644 "$WORK/web.conf" "$WEB_CONF"
    if [[ $PLATFORM == debian ]]; then
        # Migra o arquivo regular criado pelas versões 1.1.x para sites-available.
        if [[ -e $WEB_LINK && ! -L $WEB_LINK ]]; then rm -- "$WEB_LINK"; fi
        run 'Habilitando o site pelo mecanismo padrão do Debian' a2ensite 000-000-gs-linux.conf
    fi
    # Remove apenas o diretório pertencente ao assistente, já incluído no backup.
    rm -rf -- "$WEB_ROOT"
    cp -a "$WORK/site" "$WEB_ROOT"
    find "$WEB_ROOT" -type d -exec chmod 755 {} +
    find "$WEB_ROOT" -type f -exec chmod 644 {} +
    if [[ $PLATFORM == rocky ]]; then
        run 'Aplicando os contextos padrão do SELinux' restorecon -RF "$DNS_CONF" "$(dirname "$ZONE_FILE")"
"$WEB_CONF" "$WEB_ROOT"
    fi
    run 'Validando BIND e carregamento da zona' named-checkconf -z "$DNS_CONF"
    if [[ $PLATFORM == debian ]]; then run 'Validando Apache' apache2ctl configtest; else run 'Validando Apache'
httpd -t; fi
}
configure_firewall() {
    local zone service iface ufw_status rules
    if systemctl is-active --quiet firewalld; then
        iface=$(ip -o -4 addr show | awk -v wanted="$SERVER_IP" '{split($4,a,"/"); if(a[1]==wanted){print
$2;exit}}')
        zone=$(firewall-cmd --get-zone-of-interface="$iface" 2>/dev/null) || true
        if [[ -z $zone || $zone == 'no zone' ]]; then zone=$(firewall-cmd --get-default-zone); fi
        for service in dns http; do
            if ! firewall-cmd --zone="$zone" --query-service="$service" >/dev/null; then
                FW_ADDED+=("runtime|$zone|$service")
                run "Liberando $service no firewalld ($zone)" firewall-cmd --zone="$zone" --add-service="$service"
            fi
            if ! firewall-cmd --permanent --zone="$zone" --query-service="$service" >/dev/null; then
                FW_ADDED+=("permanent|$zone|$service")
                run "Persistindo $service no firewalld ($zone)" firewall-cmd --permanent --zone="$zone" --add-
service="$service"
            fi
        done
    elif command -v ufw >/dev/null && ufw_status=$(LC_ALL=C ufw status) && [[ $ufw_status == *'Status: active'* ]];
 then
        for service in 53/udp 53/tcp 80/tcp; do
            if ! grep -Eq
"^${SERVER_IP//./\\.}[[:space:]]+${service}[[:space:]]+ALLOW[[:space:]]+Anywhere[[:space:]]*$" <<<"$ufw_status";
then
                # Regra para este IPv4: não altera regras IPv6 preexistentes.
                FW_ADDED+=("ufw|$SERVER_IP|$service")
                run "Liberando $service no UFW" ufw insert 1 allow to "$SERVER_IP" port "${service%/*}" proto
"${service#*/}"
            fi
        done
    else
        # Não apagamos regras de um firewall personalizado nem iniciamos um firewall novo.
        if command -v nft >/dev/null; then
            rules=$(nft list ruleset) || die 'Não foi possível inspecionar as regras nftables.'
            [[ -z $rules ]] || die 'Há regras nftables personalizadas. Este assistente suporta firewalld ou UFW
ativos; revise a VM.'
        fi
        if command -v iptables-save >/dev/null; then
            rules=$(iptables-save) || die 'Não foi possível inspecionar iptables.'
            if grep -Eq '^-A |^:[^ ]+ DROP ' <<<"$rules"; then die 'Há regras iptables personalizadas. Revise o
firewall da VM antes de continuar.'; fi
        fi
        warn 'Nenhum firewall gerenciado ativo. Nenhuma regra foi apagada e nenhum firewall foi iniciado.'
    fi
}
verify_dns() {
    local name=$1 type=$2 expected=$3 answer
    answer=$(dig @"$SERVER_IP" "$name" "$type" +short +time=2 +tries=1)
    grep -Fxq -- "$expected" <<<"$answer" || die "DNS inesperado para $name $type. Esperado: $expected; recebido:
$answer"
}
verify_services() {
    local i name expected response
    systemctl is-active --quiet "$DNS_SERVICE" || die 'O BIND não está ativo.'
    systemctl is-active --quiet "$WEB_SERVICE" || die 'O Apache não está ativo.'
```

```
    verify_dns "ns1.$DOMAIN" A "${PRIMARY_IP:-$SERVER_IP}"
    verify_dns "$DOMAIN" NS "ns1.$DOMAIN."
    response=$(dig @"$SERVER_IP" "$DOMAIN" SOA +tcp +short +time=2 +tries=1)
    [[ $response == "ns1.$DOMAIN. hostmaster.$DOMAIN. "* ]] || die 'A consulta DNS por TCP falhou.'
    response=$(dig @"$SERVER_IP" "$DOMAIN" SOA +norecurse +time=2 +tries=1)
    [[ $response == *'status: NOERROR'* && $response =~ flags:.*[[:space:]]aa[\;\ ] ]] || die 'O DNS não respondeu
como autoridade da zona.'
    for i in "${!REC_NAME[@]}"; do
        name=${REC_NAME[i]}; [[ $name == @ ]] && name=$DOMAIN || name=$name.$DOMAIN
        expected=${REC_VALUE[i]}
        case ${REC_TYPE[i]} in
            CNAME) expected+=. ;;
            MX) expected="${REC_PRIORITY[i]} $expected." ;;
            TXT) expected=${expected//\\/\\\\}; expected=${expected//\"/\\\"}; expected="\"$expected\"" ;;
        esac
        verify_dns "$name" "${REC_TYPE[i]}" "$expected"
    done
    if [[ -n $REVERSE_ZONE ]]; then
        response=$(dig @"$SERVER_IP" "$REVERSE_ZONE" SOA +tcp +short +time=2 +tries=1)
        [[ $response == "ns1.$DOMAIN. hostmaster.$DOMAIN. "* ]] || die 'A zona reversa não está respondendo.'
        build_ptr_records
        for i in "${!PTR_IP[@]}"; do
            response=$(dig @"$SERVER_IP" -x "${PTR_IP[i]}" +short +time=2 +tries=1)
            grep -Fxq -- "${PTR_NAME[i]}" <<<"$response" || die "PTR incorreto para ${PTR_IP[i]}."
        done
        if [[ $DNS_ROLE == secondary ]]; then
            [[ -s $ZONE_FILE && -s $REVERSE_FILE ]] || die 'O secundário ainda não gravou as duas zonas
transferidas.'
            for name in "$DOMAIN" "$REVERSE_ZONE"; do
                expected=$(dig -b "$SERVER_IP" @"$PRIMARY_IP" "$name" SOA +short +time=2 +tries=1)
                response=$(dig @"$SERVER_IP" "$name" SOA +short +time=2 +tries=1)
                [[ -n $expected && $response == "$expected" ]] || die "A zona $name do secundário difere do
primário ou o primário está indisponível."
            done
        fi
    fi
    # --noproxy impede que os testes locais sejam enviados a um proxy corporativo.
    response=$(curl --noproxy '*' --fail --silent --show-error --max-time 10 --resolve "$DOMAIN:80:$SERVER_IP"
"http://$DOMAIN/gs-health.txt")
    [[ $response == "$DOMAIN" ]] || die 'O virtual host não está entregando o site configurado.'
    response=$(curl --noproxy '*' --fail --silent --show-error --max-time 10 "http://$SERVER_IP/gs-health.txt")
    [[ $response == "$DOMAIN" ]] || die 'O template não é a página padrão por IP.'
    curl --noproxy '*' --fail --silent --show-error --max-time 10 "http://$SERVER_IP/" -o "$WORK/served.html"
    cmp -s "$WEB_ROOT/index.html" "$WORK/served.html" || die 'O HTML servido não corresponde ao template
instalado.'
    ok 'DNS UDP/TCP, autoridade da zona, registros e página padrão HTTP passaram nos testes locais.'
}
wait_secondary_transfer() {
    [[ $DNS_ROLE == secondary ]] || return 0
    local deadline=$((SECONDS+45)) zone expected answer ready
    info 'Aguardando até 45 segundos pela transferência das zonas direta e reversa...'
    while (( SECONDS < deadline )); do
        ready=1
        for zone in "$DOMAIN" "$REVERSE_ZONE"; do
            expected=$(dig -b "$SERVER_IP" @"$PRIMARY_IP" "$zone" SOA +short +time=1 +tries=1 2>/dev/null) ||
expected=''
            answer=$(dig @"$SERVER_IP" "$zone" SOA +short +time=1 +tries=1 2>/dev/null) || answer=''
            [[ $expected == "ns1.$DOMAIN. hostmaster.$DOMAIN. "* && $answer == "$expected" ]] || ready=0
        done
        if (( ready )) && [[ -s $ZONE_FILE && -s $REVERSE_FILE ]]; then ok 'As duas zonas foram recebidas e os SOAs
 coincidem com o primário.'; return; fi
        sleep 1
    done
    die "Transferência não confirmada. No primário, autorize $SERVER_IP e confira domínio, zona reversa, firewall
TCP/53 e journalctl -u named."
}
save_state() {
    { printf '%s\n' GS_STATE_V2 "$DOMAIN" "$SERVER_IP" "$DNS_ROLE" "${PRIMARY_IP:-$SERVER_IP}" "${SECONDARY_IP:--}"
 "${REVERSE_NETWORK:--}" "${WEB_IP:--}" "${SERVER_IFACE:--}"; local i
      for i in "${!REC_NAME[@]}"; do printf '%s\t%s\t%s\t%s\n' "${REC_NAME[i]}" "${REC_TYPE[i]}"
"${REC_PRIORITY[i]:--}" "${REC_VALUE[i]}"; done
    } >"$STATE_DIR/state"
    chmod 600 "$STATE_DIR/state"
}
load_state() {
    [[ -f $STATE_DIR/state ]] || die 'Ainda não existe instalação registrada. Execute sudo bash gs-linux.sh.'
    local name type priority value first network
    DNS_ROLE=primary; PRIMARY_IP=''; SECONDARY_IP=''; WEB_IP=''; SERVER_IFACE=''
    REVERSE_NETWORK=''; REVERSE_ZONE=''; REVERSE_PREFIX=''; REVERSE_FILE=''; LEGACY_STATE=0
    { IFS= read -r first
      if [[ $first == GS_STATE_V2 ]]; then
          IFS= read -r DOMAIN; IFS= read -r SERVER_IP; IFS= read -r DNS_ROLE
          IFS= read -r PRIMARY_IP; IFS= read -r SECONDARY_IP; IFS= read -r network
          IFS= read -r WEB_IP; IFS= read -r SERVER_IFACE
          [[ $SECONDARY_IP != - ]] || SECONDARY_IP=''
          [[ $WEB_IP != - ]] || WEB_IP=''
```

```
          [[ $SERVER_IFACE != - ]] || SERVER_IFACE=''
          [[ $DNS_ROLE == primary || $DNS_ROLE == secondary ]] || die 'Papel DNS salvo inválido.'
          valid_ipv4 "$PRIMARY_IP" || die 'IP do primário salvo inválido.'
          [[ -z $SECONDARY_IP ]] || valid_ipv4 "$SECONDARY_IP" || die 'IP do secundário salvo inválido.'
          if [[ $network != - ]]; then set_reverse_network "$network" || die 'Rede reversa salva inválida.'; fi
      else DOMAIN=$first; IFS= read -r SERVER_IP; PRIMARY_IP=$SERVER_IP; LEGACY_STATE=1; fi
      if ! valid_domain "$DOMAIN" || ! valid_ipv4 "$SERVER_IP"; then die 'Estado salvo inválido.'; fi
      REC_NAME=(); REC_TYPE=(); REC_PRIORITY=(); REC_VALUE=()
      while IFS=$'\t' read -r name type priority value; do
          REC_NAME+=("$name"); REC_TYPE+=("$type"); REC_PRIORITY+=("${priority/-/}"); REC_VALUE+=("$value")
      done
    } <"$STATE_DIR/state"
    [[ -z $PLATFORM ]] || set_zone_paths
}

# Painel de leitura: cada teste produz evidência, sem reiniciar ou instalar serviços.
# Um erro não interrompe os demais testes. 0 = completo localmente; 1 = falhas;
# 2 = há verificações incompletas/avisos. O acesso de outra máquina é sempre separado.
REPORT_OK=0 REPORT_FAIL=0 REPORT_UNKNOWN=0
PROBE_OUTPUT='' PROBE_RC=0
report_item() {
    local status=$1 label=$2 detail=${3:-} color
    case $status in
        OK) color=$GREEN; REPORT_OK=$((REPORT_OK+1)) ;;
        FALHA) color=$RED; REPORT_FAIL=$((REPORT_FAIL+1)) ;;
        *) color=$YELLOW; REPORT_UNKNOWN=$((REPORT_UNKNOWN+1)) ;;
    esac
    printf '  %s[%s]%s %s\n' "$color$BOLD" "$status" "$RESET" "$label"
    if [[ -n $detail ]]; then printf '    %s%s%s\n' "$MUTED" "$detail" "$RESET"; fi
}
capture_probe() {
    PROBE_OUTPUT='' PROBE_RC=0
    printf '\n%s  $' "$BLUE"
    printf ' %q' "$@"
    printf '%s\n' "$RESET"
    if ! command -v "$1" >/dev/null; then PROBE_OUTPUT="Ferramenta ausente: $1"; PROBE_RC=127
    elif [[ $(type -t "$1") != function ]] && command -v timeout >/dev/null; then
        PROBE_OUTPUT=$(LC_ALL=C timeout 20s "$@" 2>&1) || PROBE_RC=$?
    else PROBE_OUTPUT=$(LC_ALL=C "$@" 2>&1) || PROBE_RC=$?; fi
    # Limita o tamanho visual, mas a decisão usa a saída completa e o status real.
    if [[ -n $PROBE_OUTPUT ]]; then
        printf '%s\n' "$PROBE_OUTPUT" | awk 'NR<=35 {print "    " $0} NR==36 {print "    [saída visual
abreviada]"}'
    fi
}
probe_failed() {
    if (( PROBE_RC == 127 || PROBE_RC == 126 )); then
        report_item 'NÃO VERIFICADO' "$1" "$PROBE_OUTPUT"; return 0
    elif (( PROBE_RC != 0 && EUID != 0 )) && [[ $PROBE_OUTPUT == *'Permission denied'* || $PROBE_OUTPUT == *'Access
 denied'* || $PROBE_OUTPUT == *'Operation not permitted'* ]]; then
        report_item 'NÃO VERIFICADO' "$1" 'Permissão insuficiente. Execute o painel com sudo.'; return 0
    elif (( PROBE_RC != 0 )); then
        report_item FALHA "$1" "Comando terminou com código $PROBE_RC (124 indica tempo esgotado)."; return 0
    fi
    return 1
}
report_command() {
    local label=$1; shift
    capture_probe "$@"
    if probe_failed "$label"; then return; fi
    report_item OK "$label"
}
report_match() {
    local label=$1 expected=$2; shift 2
    capture_probe "$@"
    if probe_failed "$label"; then return; fi
    if grep -Fxq -- "$expected" <<<"$PROBE_OUTPUT"; then report_item OK "$label" "Resposta esperada: $expected"
    else report_item FALHA "$label" "Esperado: $expected. A resposta acima não contém esse valor."; fi
}
report_port() {
    local proto=$1 port=$2 process=$3 option
    if [[ $proto == TCP ]]; then option=-lntp; else option=-lnup; fi
    capture_probe ss -H "$option" "( sport = :$port )"
    if probe_failed "Porta $port/$proto"; then return; fi
    if [[ -z $PROBE_OUTPUT ]]; then report_item FALHA "Porta $port/$proto" 'Não há socket em escuta.'
    elif (( EUID != 0 )); then report_item 'NÃO VERIFICADO' "Processo na porta $port/$proto" 'Há socket em escuta;
execute com sudo para conferir o processo proprietário.'
    elif [[ $PROBE_OUTPUT == *\"$process\"* ]]; then report_item OK "Porta $port/$proto em escuta pelo $process"
    elif grep -qv 'users:(' <<<"$PROBE_OUTPUT"; then report_item 'NÃO VERIFICADO' "Processo na porta $port/$proto"
'Há socket em escuta, mas o sistema não revelou seu proprietário (pode ocorrer em contêineres ou com permissões
restritas).'
    else report_item FALHA "Porta $port/$proto" "Há outro processo em escuta; esperado: $process."; fi
}
report_firewall() {
    local iface zone service output
    heading 'FIREWALL — permissões locais observadas'
```

```
    if (( EUID != 0 )); then report_item 'NÃO VERIFICADO' 'Firewall' 'Execute com sudo para consultar todas as
regras.'; return; fi
    if command -v firewall-cmd >/dev/null; then
        capture_probe firewall-cmd --state
        if (( PROBE_RC == 0 )) && [[ $PROBE_OUTPUT == running ]]; then
            report_item OK 'firewalld ativo'
            report_command 'Zonas ativas (inspeção)' firewall-cmd --get-active-zones
            if [[ -z $SERVER_IP ]] || ! command -v ip >/dev/null; then
                report_item 'NÃO VERIFICADO' 'Zona da interface' 'Falta o IP salvo ou a ferramenta ip.'; return
            fi
            iface=$(ip -o -4 addr show | awk -v wanted="$SERVER_IP" '{split($4,a,"/");if(a[1]==wanted){print
$2;exit}}') || iface=''
            if [[ -z $iface ]]; then report_item 'NÃO VERIFICADO' 'Zona da interface' 'O IP salvo não foi
encontrado nas interfaces.'; return; fi
            capture_probe firewall-cmd --get-zone-of-interface="$iface"
            zone=$PROBE_OUTPUT
            if [[ -z $zone || $zone == 'no zone' ]]; then capture_probe firewall-cmd --get-default-zone;
zone=$PROBE_OUTPUT; fi
            if (( PROBE_RC != 0 )) || [[ ! $zone =~ ^[a-zA-Z0-9_-]+$ ]]; then
                report_item 'NÃO VERIFICADO' 'Zona da interface' 'Não foi possível determinar a zona.'; return
            fi
            for service in dns http; do
                report_command "$service permitido agora na zona $zone" firewall-cmd --zone="$zone" --query-
service="$service"
                report_command "$service salvo permanentemente na zona $zone" firewall-cmd --permanent
--zone="$zone" --query-service="$service"
                report_command "Definição do serviço $service (inspeção das portas)" firewall-cmd --info-
service="$service"
            done
            explain 'A presença dessas permissões não comprova conectividade de outro computador.'
            return
        elif [[ $PROBE_OUTPUT != *'not running'* && $PROBE_RC != 252 ]]; then
            report_item 'NÃO VERIFICADO' 'Estado do firewalld' 'A consulta falhou; confira a saída acima.'
        fi
    fi
    if command -v ufw >/dev/null; then
        capture_probe ufw status verbose
        output=$PROBE_OUTPUT
        if (( PROBE_RC != 0 )); then report_item 'NÃO VERIFICADO' 'UFW' 'Não foi possível consultar as regras.';
return; fi
        if [[ $output == *'Status: active'* ]]; then
            report_item OK 'UFW ativo (regras exibidas acima)'
            if [[ -z $SERVER_IP ]]; then report_item 'NÃO VERIFICADO' 'Permissões UFW' 'Falta o IP salvo do
servidor.'; return; fi
            for service in 53/udp 53/tcp 80/tcp; do
                if grep -Eq
"^(${SERVER_IP//./\\.}[[:space:]]+)?${service}[[:space:]]+ALLOW([[:space:]]+IN)?[[:space:]]+Anywhere[[:space:]]*$"
<<<"$output"; then
                    report_item OK "Regra UFW permite $service (inspeção)"
                else report_item 'NÃO VERIFICADO' "Permissão UFW para $service" 'Regra simples esperada não
encontrada; confira regras personalizadas e a ordem acima.'; fi
            done
            return
        fi
    fi
    report_item AVISO 'Nenhum firewalld/UFW ativo foi confirmado' 'Firewall não será marcado como funcionando. Pode
 haver regras personalizadas ou externas.'
    if command -v nft >/dev/null; then report_command 'Regras nftables (somente inspeção)' nft list ruleset; fi
    if command -v iptables-save >/dev/null; then report_command 'Regras iptables (somente inspeção)' iptables-save;
 fi
}
report_selinux() {
    [[ $PLATFORM == rocky ]] || return 0
    heading 'SELINUX — controle de acesso no Rocky'
    capture_probe getenforce
    if probe_failed 'Estado SELinux'; then return; fi
    case $PROBE_OUTPUT in
        Enforcing) report_item OK 'SELinux aplicando as regras' ;;
        Permissive|Disabled) report_item AVISO "SELinux: $PROBE_OUTPUT" 'Os testes não comprovam funcionamento com
SELinux enforcing.' ;;
        *) report_item 'NÃO VERIFICADO' 'Estado SELinux inesperado' ;;
    esac
    report_command 'Contextos SELinux dos arquivos (inspeção)' ls -Zd "$WEB_ROOT" "$ZONE_FILE" "$DNS_CONF"
}
report_zone_sync() {
    local peer=$1 zone=$2 local_soa
    capture_probe dig @"$SERVER_IP" "$zone" SOA +short +time=2 +tries=1
    if probe_failed "SOA local de $zone"; then return; fi
    local_soa=$PROBE_OUTPUT
    capture_probe dig -b "$SERVER_IP" @"$peer" "$zone" SOA +short +time=2 +tries=1
    if probe_failed "SOA de $zone no outro servidor ($peer)"; then return; fi
    if [[ $local_soa == "ns1.$DOMAIN. hostmaster.$DOMAIN. "* && $local_soa == "$PROBE_OUTPUT" ]]; then
        report_item OK "SOA/serial de $zone coincidem nos dois servidores"
    else report_item FALHA "SOA/serial de $zone diferentes ou resposta ausente" 'Confira o primário, o serial,
allow-transfer, notify, conectividade TCP/53 e os logs.'; fi
}
```

```
report_secondary_records() {
    local zone=$1 records name ttl class type value
    # AXFR é uma consulta de leitura. O primário só a permite ao IP deste secundário.
    capture_probe dig -b "$SERVER_IP" @"$PRIMARY_IP" "$zone" AXFR +noall +answer +stats +time=2 +tries=1
    if probe_failed "Leitura dos registros de $zone no primário"; then return; fi
    if [[ $PROBE_OUTPUT != *'XFR size:'* || $PROBE_OUTPUT == *'Transfer failed.'* ]]; then
        report_item FALHA "Transferência autorizada de $zone" 'Confira allow-transfer e TCP/53 no primário.';
return
    fi
    records=$PROBE_OUTPUT
    report_item OK "Transferência autorizada de $zone"
    while read -r name ttl class type value; do
        [[ $class == IN && $ttl =~ ^[0-9]+$ ]] || continue
        case $type in
            A|CNAME|MX|TXT|PTR)
                report_match "Cópia no secundário: $name $type" "$value" dig @"$SERVER_IP" "$name" "$type" +short
+time=2 +tries=1 ;;
        esac
    done <<<"$records"
}
report_lab_dns() {
    local i name
    heading 'ATIVIDADE FIAP — REVERSA, NSLOOKUP E TRANSFERÊNCIA'
    if [[ -n $REVERSE_ZONE ]]; then
        report_match 'NS da zona reversa' "ns1.$DOMAIN." dig @"$SERVER_IP" "$REVERSE_ZONE" NS +short +time=2
+tries=1
        build_ptr_records
        for i in "${!PTR_IP[@]}"; do
            report_match "PTR ${PTR_IP[i]}" "${PTR_NAME[i]}" dig @"$SERVER_IP" -x "${PTR_IP[i]}" +short +time=2
+tries=1
        done
        if [[ $DNS_ROLE == secondary ]]; then
            report_command 'Arquivo da zona direta recebido pelo secundário' test -s "$ZONE_FILE"
            report_command 'Arquivo da zona reversa recebido pelo secundário' test -s "$REVERSE_FILE"
            report_zone_sync "$PRIMARY_IP" "$DOMAIN"
            report_zone_sync "$PRIMARY_IP" "$REVERSE_ZONE"
            report_secondary_records "$DOMAIN"
            report_secondary_records "$REVERSE_ZONE"
        elif [[ -n $SECONDARY_IP ]]; then
            report_zone_sync "$SECONDARY_IP" "$DOMAIN"
            report_zone_sync "$SECONDARY_IP" "$REVERSE_ZONE"
        fi
    else report_item AVISO 'Instalação antiga sem zona reversa' 'Reexecute a instalação 1.2 para preparar a
atividade completa.'; fi
    report_command 'nslookup: NS (evidência pedida na atividade)' nslookup -timeout=2 -retry=1 -type=NS "$DOMAIN"
"$SERVER_IP"
    report_command 'nslookup: SOA (evidência pedida na atividade)' nslookup -timeout=2 -retry=1 -type=SOA "$DOMAIN"
 "$SERVER_IP"
    for i in "${!REC_NAME[@]}"; do
        [[ ${REC_TYPE[i]} == A || ${REC_TYPE[i]} == MX ]] || continue
        name=${REC_NAME[i]}; [[ $name == @ ]] && name=$DOMAIN || name=$name.$DOMAIN
        report_command "nslookup: $name ${REC_TYPE[i]}" nslookup -timeout=2 -retry=1 "-type=${REC_TYPE[i]}" "$name"
 "$SERVER_IP"
    done
    if [[ -d /run/systemd/system ]]; then
        report_command 'Logs recentes do BIND (inspeção de carregamento/transferência)' journalctl -u
"$DNS_SERVICE" -n 40 --no-pager
    else info 'Logs do systemd indisponíveis nesta sessão. Na VM: sudo journalctl -u named -n 40 --no-pager'; fi
}
evidence_panel() {
    local saved_error i name expected answer got_state=0
    REPORT_OK=0 REPORT_FAIL=0 REPORT_UNKNOWN=0
    heading 'COMPROVAÇÃO REAL — DNS + WEB'
    info "Data: $(date -Is) | Máquina: $(hostname) | Sistema: $PLATFORM"
    explain 'Este painel consulta o sistema. Não instala, corrige, reinicia nem abre portas.'
    info 'Legenda: verde = teste passou; vermelho = falhou; amarelo = aviso/não verificado.'
    DOMAIN='' SERVER_IP=''; REC_NAME=(); REC_TYPE=(); REC_VALUE=(); REC_PRIORITY=()
    if saved_error=$( (load_state) 2>&1); then
        load_state; got_state=1
        info "Domínio: $DOMAIN | IP salvo: $SERVER_IP"
        info "Papel: $DNS_ROLE | Primário: $PRIMARY_IP | Secundário: ${SECONDARY_IP:-não configurado}"
    else report_item 'NÃO VERIFICADO' 'Configuração salva do assistente' "$saved_error Execute com sudo se faltar
permissão."; fi
    heading 'CONFIGURAÇÃO E SERVIÇOS'
    report_command 'BIND: sintaxe e carregamento das zonas' named-checkconf -z "$DNS_CONF"
    if [[ $PLATFORM == debian ]]; then report_command 'Apache: configuração válida' apache2ctl configtest
    else report_command 'Apache: configuração válida' httpd -t; fi
    if command -v systemctl >/dev/null && [[ -d /run/systemd/system ]]; then
        for name in "$DNS_SERVICE" "$WEB_SERVICE"; do
            report_match "$name está ativo AGORA" active systemctl is-active "$name"
            report_match "$name está habilitado para o BOOT" enabled systemctl is-enabled "$name"
        done
    else report_item 'NÃO VERIFICADO' 'Serviços e inicialização pelo systemd' 'systemd não está disponível nesta
sessão; isso não comprova serviço ativo nem boot.'; fi
    heading 'PORTAS — o programa está aceitando conexões?'
    report_port UDP 53 named
```

```
    report_port TCP 53 named
    report_port TCP 80 "$WEB_SERVICE"
    if (( got_state )); then
        heading 'DNS — respostas reais para os registros salvos'
        if command -v ip >/dev/null; then
            capture_probe ip -o -4 addr show
            if (( PROBE_RC != 0 )); then report_item 'NÃO VERIFICADO' 'IP atual da VM'
            elif awk -v wanted="$SERVER_IP" '{split($4,a,"/");if(a[1]==wanted)found=1}END{exit !found}'
<<<"$PROBE_OUTPUT"; then report_item OK 'IP salvo pertence à VM'
            else report_item FALHA 'IP salvo não pertence à VM' 'O endereço pode ter mudado por DHCP. Reconfigure
com o IP atual.'; fi
        else report_item 'NÃO VERIFICADO' 'IP atual da VM' 'Ferramenta ip ausente.'; fi
        report_match 'NS: servidor responsável pela zona' "ns1.$DOMAIN." dig @"$SERVER_IP" "$DOMAIN" NS +short
+time=2 +tries=1
        for i in "${!REC_NAME[@]}"; do
            name=${REC_NAME[i]}; [[ $name == @ ]] && name=$DOMAIN || name=$name.$DOMAIN
            expected=${REC_VALUE[i]}
            case ${REC_TYPE[i]} in
                CNAME) expected+=. ;; MX) expected="${REC_PRIORITY[i]} $expected." ;;
                TXT) expected=${expected//\\/\\\\}; expected=${expected//\"/\\\"}; expected="\"$expected\"" ;;
            esac
            report_match "$name ${REC_TYPE[i]} (UDP)" "$expected" dig @"$SERVER_IP" "$name" "${REC_TYPE[i]}" +short
 +time=2 +tries=1 +notcp +ignore
        done
        capture_probe dig @"$SERVER_IP" "$DOMAIN" SOA +tcp +norecurse +time=2 +tries=1
        if ! probe_failed 'SOA autoritativo por TCP'; then
            if [[ $PROBE_OUTPUT == *'status: NOERROR'* && $PROBE_OUTPUT =~ flags:.*[[:space:]]aa[\;\ ] &&
$PROBE_OUTPUT == *"ns1.$DOMAIN. hostmaster.$DOMAIN."* ]]; then
                report_item OK 'SOA autoritativo por TCP'
            else report_item FALHA 'SOA autoritativo por TCP' 'Esperava NOERROR, sinalizador aa e SOA da zona
configurada.'; fi
        fi
        report_lab_dns
        heading 'HTTP — o Apache entrega o site esperado?'
        report_match 'Página de identificação por IP' "$DOMAIN" curl --noproxy '*' --fail --silent --show-error
--max-time 10 "http://$SERVER_IP/gs-health.txt"
        report_match 'VirtualHost pelo domínio (endereço indicado ao curl)' "$DOMAIN" curl --noproxy '*' --fail
--silent --show-error --max-time 10 --resolve "$DOMAIN:80:$SERVER_IP" "http://$DOMAIN/gs-health.txt"
        capture_probe curl --noproxy '*' --fail --silent --show-error --max-time 10 "http://$SERVER_IP/"
        if ! probe_failed 'Template HTML servido'; then
            answer=$(cat "$WEB_ROOT/index.html" 2>/dev/null) || answer=''
            if [[ -n $answer && $PROBE_OUTPUT == "$answer" ]]; then report_item OK 'HTML servido corresponde ao
template instalado'
            else report_item FALHA 'HTML servido difere do template instalado ou o arquivo local não pôde ser
lido'; fi
        fi
        report_command 'CSS do template acessível' curl --noproxy '*' --fail --silent --show-error --max-time 10 -I
 "http://$SERVER_IP/css/styles.css"
    else report_item 'NÃO VERIFICADO' 'Consultas DNS e HTTP do projeto' 'Faltam domínio/IP salvos. Serviços e
portas foram inspecionados separadamente.'; fi
    report_firewall
    report_selinux
    heading 'RESULTADO E COMO APRESENTAR'
    printf '%sOK: %s%s | %sFALHAS: %s%s | %sAVISOS/NÃO VERIFICADOS: %s%s\n' "$GREEN" "$REPORT_OK" "$RESET" "$RED"
"$REPORT_FAIL" "$RESET" "$YELLOW" "$REPORT_UNKNOWN" "$RESET"
    explain 'Mostre: configuração válida → serviço ativo → porta em escuta → resposta DNS/HTTP → regras do
firewall.'
    if (( got_state )); then
        info 'Acesso de outra máquina NÃO FOI TESTADO por este painel. No computador cliente, execute:'
        printf '  dig @%s %s A\n  curl --noproxy '\''*'\'' http://%s/\n' "$SERVER_IP" "$DOMAIN" "$SERVER_IP"
        info 'Para navegar pelo nome sem --resolve, o cliente também precisa consultar este DNS para a zona.'
    fi
    if (( REPORT_FAIL )); then warn 'Há falhas. Não apresente o conjunto como funcionando; use a saída de cada
teste para corrigir.'; return 1; fi
    if (( REPORT_UNKNOWN )); then warn 'Verificação parcial: os itens amarelos ainda precisam de confirmação.';
return 2; fi
    ok 'As verificações locais acima passaram. Falta demonstrar o acesso do cliente na rede.'
}
plain_evidence_panel() {
    # Variáveis locais mantêm as cores da interface após salvar o relatório.
    local RED='' GREEN='' YELLOW='' CYAN='' BLUE='' MAGENTA='' BOLD='' RESET='' MUTED=''
    evidence_panel
}
export_report() {
    local destination result=0
    destination=$(mktemp /tmp/gs-linux-relatorio.XXXXXXXX.txt)
    # Relatório fica sem sequências de cor e com permissão 600, dada pelo mktemp.
    plain_evidence_panel >"$destination" 2>&1 || result=$?
    info "Relatório salvo em: $destination"
    info 'Contém IPs, domínio, regras e resultados reais. Leia com: less CAMINHO-DO-RELATÓRIO'
    case $result in
        0) ok 'Verificações locais concluídas.' ;; 1) warn 'O relatório contém falhas.' ;;
        2) warn 'O relatório contém verificações incompletas/avisos.' ;; *) warn "O painel foi interrompido (código
 $result)." ;;
    esac
    return "$result"
```

```
}

# PONTO DE EXTENSÃO PARA A PROVA: crie funções para as tarefas solicitadas.
# Chame-as aqui. Cada função deve validar seu resultado e usar run/info/ok.
# Se alterar novos arquivos, inclua backup_path ANTES da primeira alteração.
extra_tasks() { :; }

# No Debian, su sem "-" pode manter o PATH do usuário, sem os diretórios sbin.
# a2ensite, apache2ctl e ferramentas de firewall podem estar instalados e invisíveis.
# Preserve as entradas existentes e exporte o complemento também para os subprocessos.
ensure_admin_path() {
    local directory
    for directory in /usr/local/sbin /usr/sbin /sbin; do
        if [[ :${PATH:-}: != *":$directory:"* ]]; then
            PATH=${PATH:+$PATH:}$directory
        fi
    done
    export PATH
}

main() {
    ensure_admin_path
    case ${1:-} in
        --demo) MODE=demo ;; --check) MODE=check ;; --install) MODE=install ;;
        --learn) MODE=learn ;; --tutor) MODE=tutor ;; --status) MODE=status ;; --report) MODE=report ;;
        --help|-h) help_text; return ;;
        '') ;; *) help_text; die "Opção desconhecida: $1" ;;
    esac
    (( $# <= 1 )) || die 'Use apenas uma opção por vez.'
    colors; banner
    if [[ $MODE == menu ]]; then home_menu; fi
    case $MODE in exit) return ;; learn) learning_menu; return ;; tutor) tutor_menu; return ;; esac
    detect_platform
    case $MODE in status) evidence_panel; return ;; report) export_report; return ;; esac
    if [[ $MODE == demo ]]; then
        warn 'DEMONSTRAÇÃO: nenhuma instalação, download ou alteração será feita.'
        wizard
        info 'Na instalação real: pacotes → template → backups → configuração → firewall → testes.'
        ok 'Demonstração concluída. Nenhum serviço foi instalado.'
        return
    fi
    preflight
    WORK=$(mktemp -d /tmp/gs-linux.XXXXXXXX)
    trap cleanup EXIT
    trap 'printf "[ERRO] Etapa interrompida na linha %s.\n" "$LINENO" >&2' ERR
    trap 'exit 130' INT
    trap 'exit 143' TERM
    if [[ $MODE == check ]]; then load_state; verify_services; return; fi
    wizard
    install -d -m 700 "$STATE_DIR"
    LOG=$STATE_DIR/install-$(date +%Y%m%d-%H%M%S)-$$.log
    : >"$LOG"; chmod 600 "$LOG"
    info "Log detalhado: $LOG"
    # Pacotes ficam fora da transação: desinstalá-los automaticamente poderia quebrar outros usos.
    if command -v ss >/dev/null; then check_ports; fi
    install_packages
    ip_is_local || die "O IP $SERVER_IP não pertence a esta máquina. Execute novamente e escolha o IP correto."
    check_ports
    download_template
    begin_transaction
    if [[ $DNS_ROLE == secondary ]]; then
        run 'Parando o BIND para substituir suas cópias de zonas com segurança' systemctl stop "$DNS_SERVICE"
    fi
    apply_configuration
    configure_firewall
    run 'Ativando BIND na inicialização' systemctl enable "$DNS_SERVICE"
    run 'Ativando Apache na inicialização' systemctl enable "$WEB_SERVICE"
    run 'Iniciando/reiniciando BIND' systemctl restart "$DNS_SERVICE"
    run 'Iniciando/reiniciando Apache' systemctl restart "$WEB_SERVICE"
    wait_secondary_transfer
    verify_services
    extra_tasks
    save_state
    TRANSACTION=0
    printf '\n%sPRONTO! TESTES LOCAIS CONCLUÍDOS.%s\n' "$GREEN$BOLD" "$RESET"
    info "Abra pelo IP: http://$SERVER_IP/"
    info "Para usar http://$DOMAIN/ em outro computador, ele deve consultar o DNS $SERVER_IP."
    info 'O domínio é do laboratório; não é registrado na Internet. Roteador e firewall externo não são alterados.'
    if [[ $DNS_ROLE == primary && -n $SECONDARY_IP ]]; then
        info "Primário pronto. Configure a segunda VM ($SECONDARY_IP) no modo Secundário, usando o mesmo domínio e
rede /24."
        info 'A transferência entre as VMs será comprovada pelo painel 5 depois que ambas estiverem prontas.'
    fi
    info "Backup: $BACKUP | Log: $LOG | Template: ${TEMPLATE_SOURCE:-não instalado} (SHA-256 conferido)"
    info 'Para testar novamente: sudo bash gs-linux.sh --check'
}
```

```
# Permite que os testes carreguem as funções sem iniciar uma instalação.


# Perfil específico da GS: configuração persistida, execução sem perguntas e
# reinicialização somente após DNS, Web, rede e segurança terem passado.
EXAM_PROFILE="$SCRIPT_DIR/gs-prova.conf"
EXAM_NETWORK_PENDING=0
exam_read_profile() {
    [[ -f $EXAM_PROFILE ]] || die 'Execute primeiro: sudo bash gs-prova.sh --prepare'
    mapfile -t exam_fields < "$EXAM_PROFILE"
    [[ ${#exam_fields[@]} == 3 ]] || die 'Perfil inválido: esperado interface, IPv4 e domínio.'
    EXAM_IFACE=${exam_fields[0]}; EXAM_IP=${exam_fields[1]}; EXAM_DOMAIN=${exam_fields[2]}
    [[ $EXAM_IFACE =~ ^[a-zA-Z0-9_-]+$ && -d /sys/class/net/$EXAM_IFACE ]] || die 'Interface do perfil não existe
nesta VM.'
    valid_server_ip "$EXAM_IP" || die 'IPv4 inválido no perfil.'
    valid_domain "$EXAM_DOMAIN" && [[ $EXAM_DOMAIN == *.lab ]] || die 'O domínio deve terminar em .lab.'
}
exam_prepare() {
    local default_iface suggested='' iface addr answer
    command -v nmcli >/dev/null || die 'Rocky precisa do NetworkManager/nmcli ativo.'
    ip -br -4 addr
    default_iface=$(ip -4 route show default | awk '/dev/ {for(i=1;i<=NF;i++)if($i=="dev"){print $(i+1);exit}}')
    local -a candidates=()
    while IFS=: read -r iface kind; do
        [[ $kind == ethernet && $iface != "$default_iface" ]] && candidates+=("$iface")
    done < <(nmcli -t -f DEVICE,TYPE device status)
    if (( ${#candidates[@]} == 1 )); then suggested=${candidates[0]}; fi
    read -rp "Interface Host-Only do laboratório [${suggested:-informe}]: " iface
    iface=${iface:-$suggested}
    [[ -n $iface && $iface != "$default_iface" && -d /sys/class/net/$iface ]] || die 'Escolha a interface de
laboratório, separada da NAT/rota padrão.'
    addr=$(ip -o -4 addr show dev "$iface" | awk '{split($4,a,"/");print a[1];exit}')
    suggested=192.168.56.10
    [[ -z $addr ]] || suggested=${addr%.*}.10
    read -rp "IP fixo livre /24 [$suggested]: " answer; EXAM_IP=${answer:-$suggested}
    read -rp 'Domínio com seu nome [vladmir.lab]: ' answer; EXAM_DOMAIN=${answer:-vladmir.lab}
    valid_server_ip "$EXAM_IP" || die 'IP inválido.'
    valid_domain "$EXAM_DOMAIN" && [[ $EXAM_DOMAIN == *.lab ]] || die 'Use um domínio .lab válido.'
    printf 'Interface %s; IP %s/24; domínio %s. Confirme que o IP está livre na rede Host-Only.\n' "$iface"
"$EXAM_IP" "$EXAM_DOMAIN"
    read -rp 'Salvar para Homologação e Produção? Digite SIM: ' answer
    [[ $answer == SIM ]] || return 0
    printf '%s\n' "$iface" "$EXAM_IP" "$EXAM_DOMAIN" > "$EXAM_PROFILE"
    chmod 644 "$EXAM_PROFILE"
    echo 'Perfil salvo. Copie gs-prova.sh e gs-prova.conf para a VM. Execute --exam para instalar e reiniciar após
sucesso.'
}
exam_restore_network() {
    warn 'Restaurando o perfil de rede anterior após falha.'
    [[ -z ${EXAM_NEW_UUID:-} ]] || nmcli connection delete uuid "$EXAM_NEW_UUID" || true
    [[ -z ${EXAM_OLD_UUID:-} ]] || nmcli connection up uuid "$EXAM_OLD_UUID" || true
}
exam_static_network() {
    local default_iface name
    default_iface=$(ip -4 route show default | awk '/dev/ {for(i=1;i<=NF;i++)if($i=="dev"){print $(i+1);exit}}')
    [[ $EXAM_IFACE != "$default_iface" ]] || die 'A interface escolhida passou a ser rota padrão. Revise o perfil
antes de instalar.'
    EXAM_OLD_UUID=$(nmcli -g GENERAL.CON-UUID device show "$EXAM_IFACE")
    [[ $EXAM_OLD_UUID != -- ]] || EXAM_OLD_UUID=''
    name=gs-prova-$(date +%s)-$$
    run 'Criando perfil IPv4 estático persistente' nmcli connection add type ethernet ifname "$EXAM_IFACE" con-name
 "$name" ipv4.method manual ipv4.addresses "$EXAM_IP/24" ipv4.never-default yes ipv4.ignore-auto-dns yes
ipv6.method disabled connection.autoconnect yes connection.autoconnect-priority 999
    EXAM_NEW_UUID=$(nmcli -g connection.uuid connection show "$name")
    EXAM_NETWORK_PENDING=1
    run 'Ativando IP fixo na interface do laboratório' nmcli connection up uuid "$EXAM_NEW_UUID"
    ip -o -4 addr show dev "$EXAM_IFACE" | awk '{print $4}' | grep -Fxq "$EXAM_IP/24" || die 'IP estático não foi
aplicado.'
}
# Substitui somente o assistente no script específico da prova.
# Não aceita secundário, nem habilita transferência de zonas.
wizard() {
    exam_read_profile
    DOMAIN=$EXAM_DOMAIN; SERVER_IP=$EXAM_IP; SERVER_IFACE=$EXAM_IFACE
    DNS_ROLE=primary; PRIMARY_IP=$SERVER_IP; SECONDARY_IP=''; WEB_IP=$SERVER_IP; LEGACY_STATE=0
    set_reverse_network "${SERVER_IP%.*}.0/24"
    set_zone_paths; base_records
    # main cria o log após wizard; aqui usamos um log de preparação separado.
    install -d -m 700 "$STATE_DIR"
    LOG=$STATE_DIR/prova-rede.log
    exam_static_network
}
# O enunciado exige download da Internet; falha de download impede sucesso/reboot.
use_local_template() { return 1; }
exam_verify_final() {
```

```
    [[ $(nmcli -g ipv4.method connection show uuid "$EXAM_NEW_UUID") == manual ]] || die 'Rede não está persistida
como manual.'
    systemctl is-enabled --quiet named || die 'DNS não habilitado no boot.'
    systemctl is-enabled --quiet httpd || die 'HTTP não habilitado no boot.'
    grep -Fq 'recursion no;' "$DNS_CONF" || die 'Recursão não bloqueada.'
    grep -Fq 'allow-transfer { none; };' "$DNS_CONF" || die 'Transferência não bloqueada.'
    if dig @"$SERVER_IP" "$DOMAIN" AXFR +time=3 +tries=1 | grep -q 'XFR size:'; then die 'Transferência de zona foi
 permitida.'; fi
    local headers
    headers=$(curl --noproxy '*' -fsSI "http://$SERVER_IP/")
    grep -Eiq '^Server: Apache[[:space:]\r]*$' <<< "$headers" || die 'Cabeçalho HTTP não confirma Apache sem
versão.'
    [[ $TEMPLATE_SOURCE == internet ]] || die 'Template não veio da Internet.'
}
extra_tasks() { exam_verify_final; }
exam_entry() {
    ensure_admin_path
    case ${1:-} in
        --prepare) exam_prepare ;;
        --exam)
            [[ $EUID == 0 ]] || die 'Execute com sudo.'
            source /etc/os-release
            [[ $ID == rocky && ${VERSION_ID%%.*} =~ ^(8|9|10)$ ]] || die 'Este perfil da prova exige Rocky 8, 9 ou
10.'
            command -v nmcli >/dev/null || die 'NetworkManager não disponível.'
            main --install
            EXAM_NETWORK_PENDING=0
            info 'SUCESSO: IP fixo, DNS, HTTP, template, segurança e boot verificados. Reiniciando.'
            systemctl reboot ;;
        --status) main --status ;;
        *) printf 'GS Rocky: sudo bash gs-prova.sh --prepare (salvar dados)\nDepois: sudo bash gs-prova.sh --exam
(automático, reinicia após sucesso)\nApós boot: sudo bash gs-prova.sh --status\n' ;;
    esac
}
if [[ ${BASH_SOURCE[0]} == "$0" ]]; then exam_entry "$@"; fi
```